Digital Coursework

Contents

Contents 1
1 Task1.1 3
1.1 Problems of the Original Microarchitecture and Corresponding Modifications 4
1.2 Justification for Not Using the Negative Signal 5
2 Task1.2 6
2.1 Meaning of Control Signals 6
2.2 Instruction-by-Instruction Explanation 7
3 Task1.3 8
3.1 SystemVerilog code of the sub-blocks 8
3.2 SystemVerilog testbench of the sub-blocks 14
4 Task1.4 27
4.1 riscv code 27
4.2 program description and converting 30
4.3 riscv testbench code 32
4.4 terminal result screenshot 33
4.5 gtkwave result screenshot 34
4.6 result analyze 34
5 Task1.5 34
5.1 Introduction to the Test Program 34
5.2 Terminal Result Screenshot of the Self-Designed RISC-V Processor 34
5.3 GTKWave Result Screenshot of the Self-Designed RISC-V Processor 36
5.4 Results from the Online RISC-V Simulator 37
5.5 Result Analysis 37
6 Task2.1 37
6.1 C code 37
6.2 RISC-V code 38
6.3 Machine code 39
6.4 testbench 39
6.5 result 1 40
6.6 result 2 43
6.7 result 3 46
6.8 analyze 49
7 Task2.2 49
7.1 C code 49
7.2 RISC-V code 49
7.3 Machine code 50
7.4 testbench 50
7.5 result 1 52
7.6 result 2 55
7.7 result 3 59
7.8 analyze 62

Author's Contact Information:

A Appendix 63

1 Task1.1

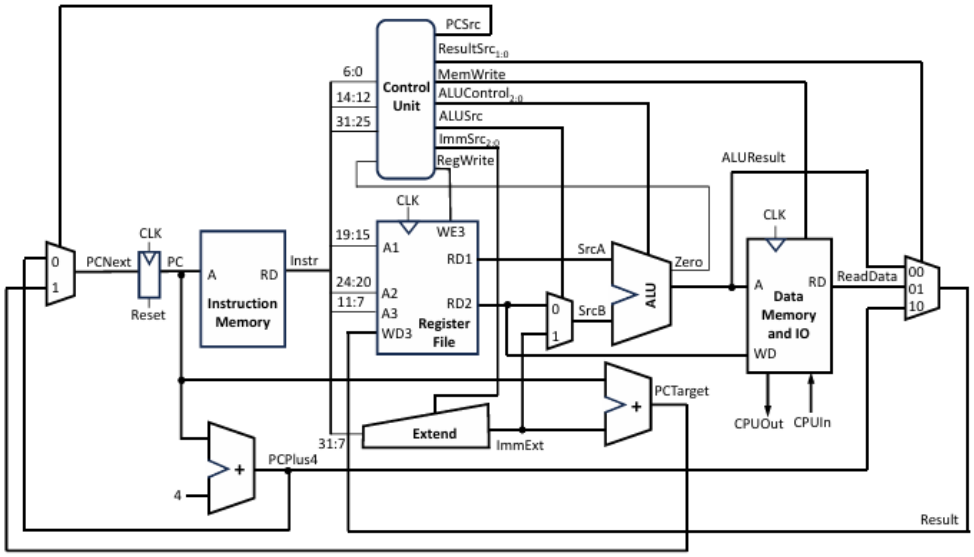


Figure 1: RISC-V microarchitecture

Fig. 1. Original circuit.

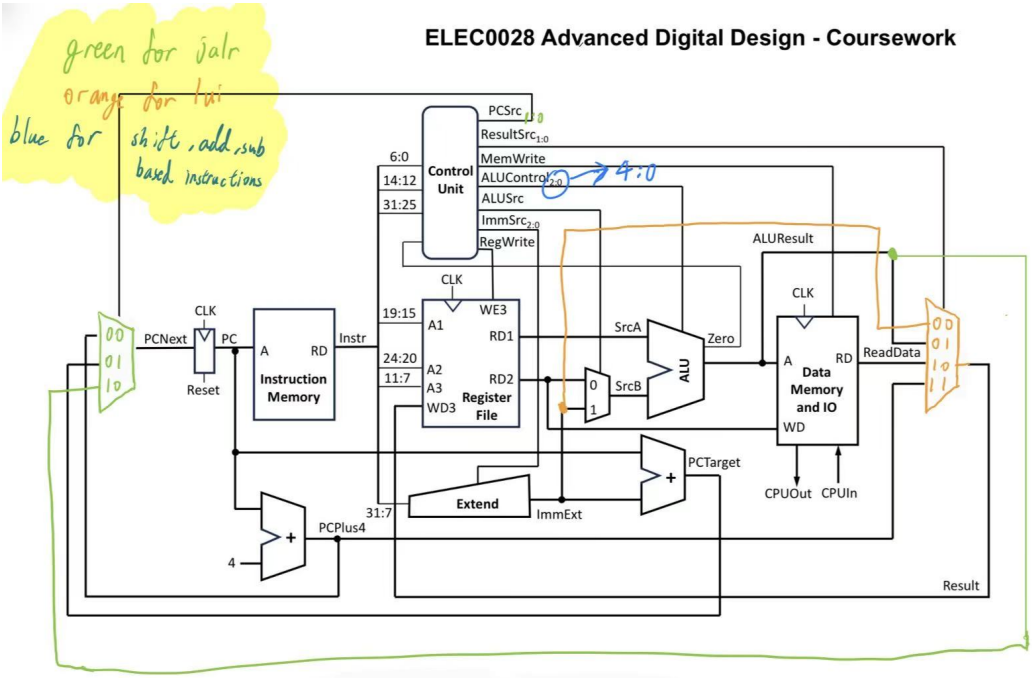


Fig. 2. Redesign of the original circuit.

1.1 Problems of the Original Microarchitecture and Corresponding Modifications

The original microarchitecture shown in Figure 1 cannot support all RV32I instructions due to several structural limitations. The key issues and corresponding modifications (shown in Figure 2) are summarised below.

1.1.1 Limitation 1: Missing PC Input(`jalr`*).* The PC multiplexer only selects between $PC + 4$ and $PC + \text{Imm}$, and does not support the $rs1 + imm$ operation required by `jalr`. As a result, `jalr` cannot be executed.

To resolve this, the PC selection logic is extended by introducing a third input from `ALUResult`, which is $rs1 + imm$, and expanding `PCSrc` to 2 bits.

1.1.2 Limitation 2: Immediate Cannot Be Written Back (`lui`*).* The output of the Extend unit (`ImmExt`) is not connected to the write-back path, and therefore cannot be stored in the register file. This prevents execution of the `lui` instruction.

This is addressed by adding `ImmExt` as an additional input to the Result multiplexer and extending `ResultSrc` accordingly.

1.1.3 Limitation 3: Insufficient ALU Control Width (`all operations that rely on addition, subtraction or shift functionality`*).* The ALU requires a 5-bit control signal to support all operations, but only a 3-bit control signal is provided in the original design. This leads to incomplete or incorrect execution of multiple instructions, including arithmetic operations (`add`, `sub`, `addi`), shift operations (`sll`, `srl`, `slli`, `srli`), comparison operations (`slt`, `slti`), memory access instructions (`lw`, `sw`), and control flow instructions (`beq`, `bne`, `blt`, `bge`, `jalr`). In particular,

all operations that rely on addition/subtraction or shift functionality may behave incorrectly due to insufficient ALU control signals.

To fix this, the `ALUControl` signal is expanded from 3 bits to 5 bits, allowing full control of all ALU functionalities.

1.2 Justification for Not Using the Negative Signal

Although the ALU shown in Figure 2 provides both `Zero` and `Negative` outputs, the `Negative` signal is not required in the redesigned processor. The reason is that all comparison-related instructions in the required instruction subset can already be implemented using the `SLT` operation together with the `Zero` signal. :contentReference[oaicite:2]index=2

For `slt` and `slti`, the ALU directly produces a result of either 1 or 0, which is then written back to the destination register. Therefore, these instructions do not require a separate sign output.

For branch instructions, the `ALURESULT` does not need to be written back, so only the branch decision matters. This means the `SLT` result can be used indirectly through the `Zero` flag.

For `blt`, the ALU is configured to perform the `SLT` operation:

- if $rs1 < rs2$, then the SLT result is 1, so `Zero` = 0, and the branch should be taken;
- if $rs1 \geq rs2$, then the SLT result is 0, so `Zero` = 1, and the branch should not be taken.

Therefore, for `blt`, the branch decision can be implemented using:

$$\texttt{PCSrc} = \{\texttt{0}, \sim \texttt{Zero}\}$$

Similarly, for `bge`:

- if $rs1 < rs2$, then the SLT result is 1, so `Zero` = 0, and the branch should not be taken;
- if $rs1 \geq rs2$, then the SLT result is 0, so `Zero` = 1, and the branch should be taken.

Hence, for `bge`, the branch decision can be implemented as:

$$\texttt{PCSrc} = \{\texttt{0}, \texttt{Zero}\}$$

This shows that `Zero` is sufficient to distinguish both branch conditions once the ALU is operating in `SLT` mode. As a result, the `Negative` output does not provide any essential additional information for the supported instruction set.

In summary, the `Negative` signal can be omitted. In the following sections, the processor design presented in this report consistently adopts the implementation without the `Negative` signal. The simulation results in Section 3.2, 4 and 5 confirm that all required instructions operate correctly under this design, demonstrating that the `Negative` output is not necessary.

2 Task1.2

instr	RegWrite	ImmSrc	ALUSrc	ALUControl	MemWrite	ResultSrc	PCSrc
add	1	XXX	0	X0X10	0	01	00
sub	1	XXX	0	X1X10	0	01	00
or	1	XXX	0	XX111	0	01	00
and	1	XXX	0	XX011	0	01	00
sll	1	XXX	0	0XX00	0	01	00
srl	1	XXX	0	1XX00	0	01	00
slt	1	XXX	0	X1X01	0	01	00
addi	1	000	1	X0X10	0	01	00
ori	1	000	1	XX111	0	01	00
andi	1	000	1	XX011	0	01	00
slli	1	000	1	0XX00	0	01	00
srli	1	000	1	1XX00	0	01	00
slti	1	000	1	X1X01	0	01	00
lw	1	000	1	X0X10	0	10	00
sw	0	001	1	X0X10	1	XX	00
beq	0	010	0	X1X10	0	XX	{0,Zero}
bne	0	010	0	X1X10	0	XX	{0,~Zero}
blt	0	010	0	X1X01	0	XX	{0,~Zero}
bge	0	010	0	X1X01	0	XX	{0,Zero}
jal	1	100	X	XXXXX	0	11	01
jalr	1	000	1	X0X10	0	11	10
lui	1	011	X	XXXXX	0	00	00

Fig. 3. Control signals for supported instructions.

Figure 3 summarises the control signals for all supported instructions. Each signal is defined as follows.

2.1 Meaning of Control Signals

RegWrite enables writing the Result back to register `rd`.

ImmSrc selects the type of immediate generated by the Extend unit:

- 000: I-type
- 001: S-type
- 010: B-type
- 011: U-type
- 100: J-type

ALUSrc selects the second ALU operand:

- 0: register value (RD2)
- 1: immediate (ImmExt)

ALUControl[4:0] determines the ALU operation:

- Bit [4]: shift direction (0 = left, 1 = right)
- Bit [3]: arithmetic (0 = add, 1 = sub)
- Bit [2]: logic (0 = and, 1 = or)
- Bits [1:0]:
  - 00: shift
  - 01: set-less-than
  - 10: arithmetic

  - 11: logic

MemWrite enables writing to data memory.
ResultSrc selects the write-back source:

- 00: ImmExt
- 01: ALUResult
- 10: ReadData
- 11: PCPlus4

PCSrc selects the next PC:

- 00: $PC + 4$
- 01: PC + ImmExt
- 10: ALUResult

For branch instructions, symbolic forms are used:

- {0,Zero}: branch when Zero = 1
- {0,~Zero}: branch when Zero = 0

2.2 Instruction-by-Instruction Explanation

2.2.1 R-type Arithmetic and Logic Instructions (`add`*,* `sub`*,* `and`*,* `or`*).* These instructions use two register operands, so `ALUSrc = 0` and `RegWrite = 1`. The result is written back from the ALU (`ResultSrc = 01`), and execution proceeds sequentially (`PCSrc = 00`).

The difference lies in `ALUControl`: `add` uses addition (`X0X10`), `sub` uses subtraction (`X1X10`), while `and` and `or` use the logic path (`XX011` and `XX111`, respectively).

2.2.2 R-type Shift and Comparison Instructions (`sll`*,* `srl`*,* `slt`*).* These instructions also use register operands (`ALUSrc = 0`, `RegWrite = 1`, `ResultSrc = 01`).

Shift instructions (`sll`, `srl`) select the shift path (`[1:0] = 00`), with bit [4] determining direction (left or right).

`slt` selects the comparison path (`[1:0] = 01`), producing a result of 0 or 1.

2.2.3 I-type Arithmetic and Logic Instructions (`addi`*,* `andi`*,* `ori`*).* These instructions use an immediate operand, so `ImmSrc = 000` and `ALUSrc = 1`. The ALU result is written back to the register file.

The ALU operation matches their R-type counterparts: addition for `addi`, AND for `andi`, and OR for `ori`.

2.2.4 I-type Shift and Comparison Instructions (`slli`*,* `srli`*,* `slti`*).* These instructions also use `ImmExt` as the second operand (`ALUSrc = 1`).

Shift instructions select the shift path, using the lower 5 bits of the immediate as the shift amount.

`slti` performs comparison using the SLT path and writes the result (0 or 1) to the destination register.

2.2.5 Memory Instructions (`lw`*,* `sw`*).* Both instructions compute the effective address using addition (`rs1 + ImmExt`), so `ALUSrc = 1` and `ALUControl` selects addition.

For `lw`, `RegWrite = 1` and `ResultSrc = 10` so that data read from memory is written to the register file.

For `sw`, `RegWrite = 0` and `MemWrite = 1`, storing the value of `rs2` into memory.

2.2.6 Branch Instructions (`beq`*,* `bne`*,* `blt`*,* `bge`*).* These instructions do not write to the register file (`RegWrite = 0`) and use B-type immediates.

`beq` and `bne` use subtraction to compare operands, with branching controlled by the `Zero` signal.

blt and bge use the SLT operation. The branch decision is determined from the Zero flag: blt branches when Zero = 0, while bge branches when Zero = 1.

More detailed reasoning for this design choice is discussed in Section 1.2.

*2.2.7 Jump Instructions (*jal, jalr*).* Both instructions write $PC + 4$ to the destination register (ResultSrc = 11, RegWrite = 1).

jal uses a J-type immediate and sets PCSrc = 01, selecting PC + ImmExt.

jalr uses an I-type immediate and computes the target address using the ALU (rs1 + ImmExt), so PCSrc = 10.

*2.2.8 Immediate Upper Instruction (*lui*).* lui directly writes the U-type immediate to the destination register, so ResultSrc = 00. The ALU is not used, and execution proceeds sequentially.

3 Task1.3

3.1 SystemVerilog code of the sub-blocks

3.1.1 Program Counter. The SystemVerilog implementation of the Program Counter module is shown below.

Listing 1. Program Counter module

```
module program_counter (
    input  logic        CLK,
    input  logic        reset,
    input  logic [31:0] PCNext,
    output logic [31:0] PC
);

always_ff @(posedge CLK) begin
    if (reset)
        PC <= 32'h00000000;
    else
        PC <= PCNext;
end

endmodule
```

3.1.2 Extend. The SystemVerilog implementation of the Extend module is shown below.

Listing 2. Extend module

```
module extend (
    input  logic [31:0] Instr,      // Full 32-bit instruction
    input  logic [2:0]  ImmSrc,     // Immediate type selector
    output logic [31:0] ImmExt      // Extended immediate
);

always_comb begin
    case (ImmSrc)
        3'b000: begin
            // I-type immediate
            ImmExt = {{20{Instr[31]}}, Instr[31:20]};
```

```
        end

        3'b001: begin
            // S-type immediate
            ImmExt = {{20{Instr[31]}}, Instr[31:25], Instr[11:7]};
        end

        3'b010: begin
            // B-type immediate
            ImmExt = {{19{Instr[31]}}, Instr[31], Instr[7], Instr[30:25], Instr[11:8], 1'b0};
        end

        3'b011: begin
            // U-type immediate
            ImmExt = {Instr[31:12], 12'b0};
        end

        3'b100: begin
            // J-type immediate
            ImmExt = {{12{Instr[31]}}, Instr[19:12], Instr[20], Instr[30:21], 1'b0};
        end

        default: begin
            ImmExt = 32'h00000000;
        end

    endcase
end

endmodule
```

3.1.3 Arithmetic Logic Unit. The SystemVerilog implementation of the Arithmetic Logic Unit module is shown below.

Listing 3. Arithmetic Logic Unit module

```
module alu (
    input  logic [31:0] A,             // ALU input A
    input  logic [31:0] B,             // ALU input B
    input  logic [4:0]  ALUControl,    // ALU control signal
    output logic [31:0] ALUResult,     // ALU output result
    output logic        Zero          // Zero flag
);

    logic [31:0] shift_result;
    logic [31:0] arith_result;
    logic [31:0] logic_result;
    logic [31:0] slt_result;


```

```
    always_comb begin

        // Shift path
        // ALUControl[4] = 0 for SLL, 1 for SRL
        case (ALUControl[4])
            1'b0: shift_result = A << B[4:0];
            1'b1: shift_result = A >> B[4:0];
        endcase

        // Arithmetic path
        // ALUControl[3] = 0 for add, 1 for subtract
        case (ALUControl[3])
            1'b0: arith_result = A + B;
            1'b1: arith_result = A - B;
        endcase

        // Logic path
        // ALUControl[2] = 0 for AND, 1 for OR
        case (ALUControl[2])
            1'b0: logic_result = A & B;
            1'b1: logic_result = A | B;
        endcase

        // Signed comparison must not rely only on the subtraction sign bit,
        // because subtraction can overflow when operands have different signs.
        slt_result = {31'b0, $signed(A) < $signed(B)};

        // Main ALU output multiplexer
        case (ALUControl[1:0])
            2'b00: ALUResult = shift_result;
            2'b01: ALUResult = slt_result;
            2'b10: ALUResult = arith_result;
            2'b11: ALUResult = logic_result;
            default: ALUResult = 32'h0;
        endcase
    end

    // Status flags
    assign Zero     = (ALUResult == 32'h0);

endmodule
```

3.1.4 Control Unit. The SystemVerilog implementation of the Control Unit module is shown below.

Listing 4. Control Unit module

```
module control_unit (
    input  logic [31:0] instr,
    input  logic        Zero,

    output logic        RegWrite,
```

```
    output logic [2:0]  ImmSrc,
    output logic        ALUSrc,
    output logic [4:0]  ALUControl,
    output logic        MemWrite,
    output logic [1:0]  ResultSrc,
    output logic [1:0]  PCSrc
);

    logic [6:0] opcode;
    logic [2:0] funct3;
    logic [6:0] funct7;

    assign opcode = instr[6:0];
    assign funct3 = instr[14:12];
    assign funct7 = instr[31:25];

    always_comb begin
        // default values
        RegWrite   = 1'b0;
        ImmSrc     = 3'b000;
        ALUSrc     = 1'b0;
        ALUControl = 5'b00000;
        MemWrite   = 1'b0;
        ResultSrc  = 2'b00;
        PCSrc      = 2'b00;

        case (opcode)

            // ========================
            // R-type: add, sub, or, and, sll, srl, slt
            // opcode = 0110011
            // ========================
            7'b0110011: begin
                RegWrite  = 1'b1;
                ALUSrc    = 1'b0;
                ResultSrc = 2'b01;
                PCSrc     = 2'b00;

                case (funct3)
                    3'b000: begin
                        if (funct7 == 7'b0100000)
                            ALUControl = 5'b01010; // sub
                        else
                            ALUControl = 5'b00010; // add
                    end
                    3'b110: ALUControl = 5'b00111; // or
                    3'b111: ALUControl = 5'b00011; // and
                    3'b001: ALUControl = 5'b00000; // sll
                    3'b101: ALUControl = 5'b10000; // srl
                    3'b010: ALUControl = 5'b01001; // slt
                    default: ALUControl = 5'b00000;
                endcase
```

```
            end

            // ========================
            // I-type arithmetic:
            // addi, ori, andi, slli, srli, slti
            // opcode = 0010011
            // ========================
            7'b0010011: begin
                RegWrite  = 1'b1;
                ImmSrc    = 3'b000;
                ALUSrc    = 1'b1;
                ResultSrc = 2'b01;
                PCSrc     = 2'b00;

                case (funct3)
                    3'b000: ALUControl = 5'b00010; // addi
                    3'b110: ALUControl = 5'b00111; // ori
                    3'b111: ALUControl = 5'b00011; // andi
                    3'b001: ALUControl = 5'b00000; // slli
                    3'b101: ALUControl = 5'b10000; // srli
                    3'b010: ALUControl = 5'b01001; // slti
                    default: ALUControl = 5'b00000;
                endcase
            end

            // ========================
            // lw
            // opcode = 0000011
            // ========================
            7'b0000011: begin
                RegWrite   = 1'b1;
                ImmSrc     = 3'b000;
                ALUSrc     = 1'b1;
                ALUControl = 5'b00010; // add for address
                MemWrite   = 1'b0;
                ResultSrc  = 2'b10;
                PCSrc      = 2'b00;
            end

            // ========================
            // sw
            // opcode = 0100011
            // ========================
            7'b0100011: begin
                RegWrite   = 1'b0;
                ImmSrc     = 3'b001;
                ALUSrc     = 1'b1;
                ALUControl = 5'b00010; // add for address
                MemWrite   = 1'b1;
                ResultSrc  = 2'b00;    // don't care
                PCSrc      = 2'b00;
            end
```

```

            // ========================
            // Branches: beq, bne, blt, bge
            // opcode = 1100011
            // ========================
            7'b1100011: begin
                RegWrite  = 1'b0;
                ImmSrc    = 3'b010;
                ALUSrc    = 1'b0;
                MemWrite  = 1'b0;
                ResultSrc = 2'b00; // don't care

                case (funct3)
                    3'b000: begin
                        ALUControl = 5'b01010;           // beq -> sub
                        PCSrc      = Zero ? 2'b01 : 2'b00;
                    end
                    3'b001: begin
                        ALUControl = 5'b01010;           // bne -> sub
                        PCSrc      = Zero ? 2'b00 : 2'b01;
                    end
                    3'b100: begin
                        ALUControl = 5'b01001;           // blt -> slt
                        PCSrc      = Zero ? 2'b00 : 2'b01;
                    end
                    3'b101: begin
                        ALUControl = 5'b01001;           // bge -> slt
                        PCSrc      = Zero ? 2'b01 : 2'b00;
                    end
                    default: begin
                        ALUControl = 5'b00010;
                        PCSrc      = 2'b00;
                    end
                endcase
            end

            // ========================
            // jal
            // opcode = 1101111
            // ========================
            7'b1101111: begin
                RegWrite   = 1'b1;
                ImmSrc     = 3'b100;
                ALUSrc     = 1'b0;      // don't care
                ALUControl = 5'b00000;  // don't care
                MemWrite   = 1'b0;
                ResultSrc  = 2'b11;     // write PC+4
                PCSrc      = 2'b01;
            end

            // ========================
            // jalr
```

```
            // opcode = 1100111
            // ========================
            7'b1100111: begin
                RegWrite   = 1'b1;
                ImmSrc     = 3'b000;
                ALUSrc     = 1'b1;
                ALUControl = 5'b00010;  // rs1 + imm
                MemWrite   = 1'b0;
                ResultSrc  = 2'b11;     // write PC+4
                PCSrc      = 2'b10;
            end

            // ========================
            // lui
            // opcode = 0110111
            // ========================
            7'b0110111: begin
                RegWrite   = 1'b1;
                ImmSrc     = 3'b011;
                ALUSrc     = 1'b0;      // don't care
                ALUControl = 5'b00000;  // don't care
                MemWrite   = 1'b0;
                ResultSrc  = 2'b00;     // immediate
                PCSrc      = 2'b00;
            end

            default: begin
                RegWrite   = 1'b0;
                ImmSrc     = 3'b000;
                ALUSrc     = 1'b0;
                ALUControl = 5'b00000;
                MemWrite   = 1'b0;
                ResultSrc  = 2'b00;
                PCSrc      = 2'b00;
            end
        endcase
    end

endmodule
```

3.2 SystemVerilog testbench of the sub-blocks

3.2.1 Program Counter. The testbench code of the Program Counter module is shown below.

Listing 5. Program Counter testbench

```
`timescale 1ns/1ps
`include "./SVcode/pc/pc.sv"

module program_counter_tb;

    logic        CLK;
```

```
    logic        reset;
    logic [31:0] PCNext;
    logic [31:0] PC;

    program_counter dut (
        .CLK(CLK),
        .reset(reset),
        .PCNext(PCNext),
        .PC(PC)
    );

    initial begin
        $dumpfile("program_counter_tb.vcd");
        $dumpvars(0, program_counter_tb);
    end

    initial begin
        CLK = 0;
        forever #5 CLK = ~CLK;
    end

    initial begin
        $display("Time\tCLK\treset\tPCNext\t\tPC");

        $monitor("%0t\t%b\t%b\t%h\t%h", $time, CLK, reset, PCNext, PC);

        reset  = 1;
        PCNext = 32'h00000004;
        #10;

        reset  = 0;
        PCNext = 32'h00000004;
        #10;

        PCNext = 32'h00000008;
        #10;

        PCNext = 32'h0000000C;
        #10;

        reset = 1;
        PCNext = 32'h00000010;
        #10;

        reset = 0;
        PCNext = 32'h00000014;
        #10;

        $finish;
    end

endmodule
```

Listing 6. Terminal result of the Program Counter module

```
VCD info: dumpfile program_counter_tb.vcd opened for output.
Time    CLK reset   PCNext      PC
0   0   1   00000004    xxxxxxxx
5000    1   1   00000004    00000000
10000   0   0   00000004    00000000
15000   1   0   00000004    00000004
20000   0   0   00000008    00000004
25000   1   0   00000008    00000008
30000   0   0   0000000c    00000008
35000   1   0   0000000c    0000000c
40000   0   1   00000010    0000000c
45000   1   1   00000010    00000000
50000   0   0   00000014    00000000
55000   1   0   00000014    00000014
60000   0   0   00000014    00000014
```

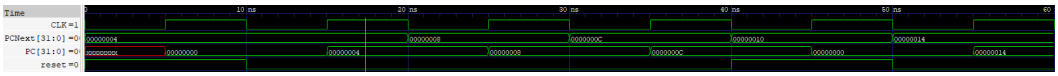


Fig. 4. GTKWave result screenshot for the Program Counter module.

The analysis of the Program Counter testbench is shown below.

The simulation results show that the Program Counter module operates correctly. Whenever reset = 1, the PC is synchronously reset to 0 at the next positive clock edge. Otherwise, the PC is updated with the value of PCNext on each positive clock edge, which matches the expected behaviour of the design.

At the beginning of simulation, the PC output appears as X. This is normal, because the register has not yet been initialised before the first active clock edge. After the first clock edge, the PC takes a definite value determined by either reset or PCNext, so the module then behaves as expected.

3.2.2 Extend. The testbench code of the Extend module is shown below.

Listing 7. Extend testbench

```
`timescale 1ns/1ps
`include "./SVcode/extend/extend.sv"

module extend_tb;

    logic [31:0] Instr;
    logic [2:0]  ImmSrc;
    logic [31:0] ImmExt;

    extend dut (
        .Instr(Instr),
        .ImmSrc(ImmSrc),
        .ImmExt(ImmExt)
```

```
    );

    initial begin
        $dumpfile("extend_tb.vcd");
        $dumpvars(0, extend_tb);
    end

    initial begin
        $display("Time\tImmSrc\tInstr\t\t\tImmExt");
        $monitor("%0t\t%b\t%h\t%h", $time, ImmSrc, Instr, ImmExt);

        // I-type immediate tests
        Instr  = 32'h00500093;   // imm = 5
        ImmSrc = 3'b000;
        #10;

        Instr  = 32'h00C00113;   // imm = 12
        ImmSrc = 3'b000;
        #10;

        Instr  = 32'hFFF00093;   // imm = -1
        ImmSrc = 3'b000;
        #10;

        Instr  = 32'h80000093;   // imm = -2048
        ImmSrc = 3'b000;
        #10;

        // S-type immediate tests
        Instr  = 32'h0020A423;   // imm = 8
        ImmSrc = 3'b001;
        #10;

        Instr  = 32'hFE20AE23;   // imm = -4
        ImmSrc = 3'b001;
        #10;

        Instr  = 32'h7E20AF23;   // positive S-type immediate
        ImmSrc = 3'b001;
        #10;

        // B-type immediate tests
        Instr  = 32'h00208863;   // imm = 16
        ImmSrc = 3'b010;
        #10;

        Instr  = 32'hFE208EE3;   // negative branch immediate
        ImmSrc = 3'b010;
        #10;

        Instr  = 32'h00000063;   // imm = 0
        ImmSrc = 3'b010;
```

```
        #10;

        Instr  = 32'h7E208FE3;   // positive branch immediate
        ImmSrc = 3'b010;
        #10;

        // U-type immediate tests
        Instr  = 32'h123450B7;   // imm = 0x12345000
        ImmSrc = 3'b011;
        #10;

        Instr  = 32'hABCDE137;   // imm = 0xABCDE000
        ImmSrc = 3'b011;
        #10;

        Instr  = 32'hFFF00037;   // imm = 0xFFF00000
        ImmSrc = 3'b011;
        #10;

        // J-type immediate tests
        Instr  = 32'h001000EF;   // small positive jump immediate
        ImmSrc = 3'b100;
        #10;

        Instr  = 32'h7FF000EF;   // large positive jump immediate
        ImmSrc = 3'b100;
        #10;

        Instr  = 32'h800000EF;   // negative jump immediate
        ImmSrc = 3'b100;
        #10;

        Instr  = 32'h000000EF;   // imm = 0
        ImmSrc = 3'b100;
        #10;

        $finish;
    end

endmodule
```

Listing 8. Terminal result of the Extend module

```
VCD info: dumpfile extend_tb.vcd opened for output.
Time    ImmSrc  Instr           ImmExt
0   000 00500093    00000005
10000   000 00c00113    0000000c
20000   000 fff00093    ffffffff
30000   000 80000093    fffff800
40000   001 0020a423    00000008
50000   001 fe20ae23    fffffffc
60000   001 7e20af23    000007fe
```

```
70000   010 00208863    00000010
80000   010 fe208ee3    fffffffc
90000   010 00000063    00000000
100000  010 7e208fe3    00000ffe
110000  011 123450b7    12345000
120000  011 abcde137    abcde000
130000  011 fff00037    fff00000
140000  100 001000ef    00000800
150000  100 7ff000ef    00000ffe
160000  100 800000ef    fff00000
170000  100 000000ef    00000000
```

Fig. 5. GTKWave result screenshot for the Extend module.

The analysis of the Extend testbench is shown below.

The simulation results show that the Extend module operates correctly. The test cases cover all required immediate formats, including I-type, S-type, B-type, U-type, and J-type instructions. Positive values, negative values, and boundary cases were all tested.

For each instruction format, the module correctly extracts the immediate field from the instruction and performs the required sign extension or bit concatenation. No incorrect or unexpected outputs were observed during simulation. Therefore, the Extend module satisfies the functional requirements of the design.

3.2.3 Arithmetic Logic Unit. The testbench code of the Arithmetic Logic Unit module is shown below.

Listing 9. Arithmetic Logic Unit testbench

```
`timescale 1ns/1ps
`include "./SVcode/alu/alu.sv"

module alu_tb;

    logic [31:0] A;
    logic [31:0] B;
    logic [4:0]  ALUControl;
    logic [31:0] ALUResult;
    logic        Zero;

    alu dut (
        .A(A),
        .B(B),
        .ALUControl(ALUControl),
        .ALUResult(ALUResult),
        .Zero(Zero)
    );

    initial begin
```

```
        $dumpfile("alu_tb.vcd");
        $dumpvars(0, alu_tb);
    end

    initial begin
        $display("Time\tALUControl\tA\t\tB\t\tALUResult\tZero");
        $monitor("%0t\t%b\t\t%h\t%h\t%h\t%b",
                 $time, ALUControl, A, B, ALUResult, Zero);

        // ========================================================
        // Shift operation tests
        // ALUControl[1:0] = 00
        // ALUControl[4]   = 0 for SLL, 1 for SRL
        // ========================================================

        // SLL: 1 << 2 = 4
        A = 32'h00000001; B = 32'h00000002; ALUControl = 5'b00000;
        #10;

        // SLL: 3 << 4 = 0x30
        A = 32'h00000003; B = 32'h00000004; ALUControl = 5'b00000;
        #10;

        // SLL: shift by 0
        A = 32'h12345678; B = 32'h00000000; ALUControl = 5'b00000;
        #10;

        // SRL: 0x10 >> 2 = 0x4
        A = 32'h00000010; B = 32'h00000002; ALUControl = 5'b10000;
        #10;

        // SRL: 0x80000000 >> 4 = 0x08000000
        A = 32'h80000000; B = 32'h00000004; ALUControl = 5'b10000;
        #10;

        // SRL: shift by 0
        A = 32'h87654321; B = 32'h00000000; ALUControl = 5'b10000;
        #10;

        // ========================================================
        // SLT operation tests
        // ALUControl[1:0] = 01
        // Arithmetic path uses subtraction when ALUControl[3] = 1
        // ========================================================

        // 3 < 5 -> result should be 1
        A = 32'h00000003; B = 32'h00000005; ALUControl = 5'b01001;
        #10;

        // 7 < 2 -> result should be 0
        A = 32'h00000007; B = 32'h00000002; ALUControl = 5'b01001;
        #10;
```

```

        // Equal values -> result should be 0
        A = 32'h00000009; B = 32'h00000009; ALUControl = 5'b01001;
        #10;

        // Negative less-than case based on subtraction sign bit
        A = 32'hFFFFFFFE; B = 32'h00000001; ALUControl = 5'b01001;
        #10;

        // ========================================================
        // Arithmetic operation tests
        // ALUControl[1:0] = 10
        // ALUControl[3]   = 0 for ADD, 1 for SUB
        // ========================================================

        // ADD: 5 + 7 = 12
        A = 32'h00000005; B = 32'h00000007; ALUControl = 5'b00010;
        #10;

        // ADD: 0 + 0 = 0
        A = 32'h00000000; B = 32'h00000000; ALUControl = 5'b00010;
        #10;

        // ADD: negative + positive
        A = 32'hFFFFFFFC; B = 32'h00000002; ALUControl = 5'b00010;
        #10;

        // SUB: 9 - 4 = 5
        A = 32'h00000009; B = 32'h00000004; ALUControl = 5'b01010;
        #10;

        // SUB: 4 - 9 = negative result
        A = 32'h00000004; B = 32'h00000009; ALUControl = 5'b01010;
        #10;

        // SUB: equal values -> 0
        A = 32'h0000000A; B = 32'h0000000A; ALUControl = 5'b01010;
        #10;

        // ========================================================
        // Logic operation tests
        // ALUControl[1:0] = 11
        // ALUControl[2]   = 0 for AND, 1 for OR
        // ========================================================

        // AND
        A = 32'hF0F0F0F0; B = 32'h0FF00FF0; ALUControl = 5'b00011;
        #10;

        // AND with zero
        A = 32'h12345678; B = 32'h00000000; ALUControl = 5'b00011;
        #10;
```

```
        // AND all ones
        A = 32'hFFFFFFFF; B = 32'hAAAAAAAA; ALUControl = 5'b00011;
        #10;

        // OR
        A = 32'hF0F0F0F0; B = 32'h0FF00FF0; ALUControl = 5'b00111;
        #10;

        // OR with zero
        A = 32'h12345678; B = 32'h00000000; ALUControl = 5'b00111;
        #10;

        // OR mixed pattern
        A = 32'h55555555; B = 32'hAAAAAAAA; ALUControl = 5'b00111;
        #10;

        $finish;
    end

endmodule
```

Listing 10. Terminal result of the Arithmetic Logic Unit module

```
VCD info: dumpfile alu_tb.vcd opened for output.
Time    ALUControl  A       B       ALUResult   Zero
0   00000       00000001    00000002    00000004    0
10000   00000       00000003    00000004    00000030    0
20000   00000       12345678    00000000    12345678    0
30000   10000       00000010    00000002    00000004    0
40000   10000       80000000    00000004    08000000    0
50000   10000       87654321    00000000    87654321    0
60000   01001       00000003    00000005    00000001    0
70000   01001       00000007    00000002    00000000    1
80000   01001       00000009    00000009    00000000    1
90000   01001       fffffffe    00000001    00000001    0
100000  00010       00000005    00000007    0000000c    0
110000  00010       00000000    00000000    00000000    1
120000  00010       fffffffc    00000002    fffffffe    0
130000  01010       00000009    00000004    00000005    0
140000  01010       00000004    00000009    fffffffb    0
150000  01010       0000000a    0000000a    00000000    1
160000  00011       f0f0f0f0    0ff00ff0    00f000f0    0
170000  00011       12345678    00000000    00000000    1
180000  00011       ffffffff    aaaaaaaa    aaaaaaaa    0
190000  00111       f0f0f0f0    0ff00ff0    fff0fff0    0
200000  00111       12345678    00000000    12345678    0
210000  00111       55555555    aaaaaaaa    ffffffff    0
```

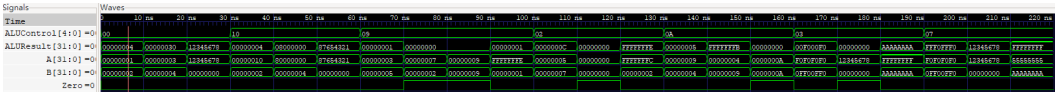


Fig. 6. GTKWave result screenshot for the Arithmetic Logic Unit module.

The analysis of the Arithmetic Logic Unit testbench is shown below.

The ALU simulation results show that the module operates correctly across all tested functional categories, including shift, arithmetic, logic, and set-less-than operations. For each ALUControl configuration, the output matches the expected result, confirming that the datapath selection and control encoding are implemented correctly.

The Zero flag is also asserted correctly when the ALU result is equal to zero, which is important for branch decision logic. In particular, branch instructions such as BLT and BGE can be supported using the SLT operation together with the Zero flag, without requiring a separate Negative signal. Overall, the ALU behaves as intended and satisfies the coursework requirements under the tested conditions.

3.2.4 Control Unit. The testbench code of the Control Unit module is shown below.

Listing 11. Control Unit testbench

```
`timescale 1ns/1ps
`include "./SVcode/control_unit/control_unit.sv"

module control_unit_tb;

    logic [31:0] instr;
    logic        Zero;

    logic        RegWrite;
    logic [2:0]  ImmSrc;
    logic        ALUSrc;
    logic [4:0]  ALUControl;
    logic        MemWrite;
    logic [1:0]  ResultSrc;
    logic [1:0]  PCSrc;

    control_unit dut (
        .instr(instr),
        .Zero(Zero),
        .RegWrite(RegWrite),
        .ImmSrc(ImmSrc),
        .ALUSrc(ALUSrc),
        .ALUControl(ALUControl),
        .MemWrite(MemWrite),
        .ResultSrc(ResultSrc),
        .PCSrc(PCSrc)
    );

    initial begin
        $dumpfile("control_unit_tb.vcd");
        $dumpvars(0, control_unit_tb);
```

```
    end

    initial begin
        $display("Time\tinstr\t\t\tZero\tRegWrite ImmSrc ALUSrc ALUCtrl MemWrite
    ↪ ResultSrc PCSrc");
        $monitor("%0t\t%h\t%b\t%b\t %b\t   %b\t   %b\t   %b\t    %b\t    %b",
                 $time, instr, Zero, RegWrite, ImmSrc, ALUSrc, ALUControl,
                 MemWrite, ResultSrc, PCSrc);

        // ========================================================
        // R-type instruction tests
        // ========================================================

        // add  x1, x2, x3
        instr = 32'b0000000_00011_00010_000_00001_0110011; Zero = 0;
        #10;

        // sub  x1, x2, x3
        instr = 32'b0100000_00011_00010_000_00001_0110011; Zero = 0;
        #10;

        // or   x1, x2, x3
        instr = 32'b0000000_00011_00010_110_00001_0110011; Zero = 0;
        #10;

        // and  x1, x2, x3
        instr = 32'b0000000_00011_00010_111_00001_0110011; Zero = 0;
        #10;

        // sll  x1, x2, x3
        instr = 32'b0000000_00011_00010_001_00001_0110011; Zero = 0;
        #10;

        // srl  x1, x2, x3
        instr = 32'b0000000_00011_00010_101_00001_0110011; Zero = 0;
        #10;

        // slt  x1, x2, x3
        instr = 32'b0000000_00011_00010_010_00001_0110011; Zero = 0;
        #10;

        // ========================================================
        // I-type arithmetic instruction tests
        // ========================================================

        // addi x1, x2, 5
        instr = 32'b000000000101_00010_000_00001_0010011; Zero = 0;
        #10;

        // ori  x1, x2, 5
        instr = 32'b000000000101_00010_110_00001_0010011; Zero = 0;
        #10;
```

```

        // andi x1, x2, 5
        instr = 32'b000000000101_00010_111_00001_0010011; Zero = 0;
        #10;

        // slli x1, x2, 3
        instr = 32'b0000000_00011_00010_001_00001_0010011; Zero = 0;
        #10;

        // srli x1, x2, 3
        instr = 32'b0000000_00011_00010_101_00001_0010011; Zero = 0;
        #10;

        // slti x1, x2, 5
        instr = 32'b000000000101_00010_010_00001_0010011; Zero = 0;
        #10;

        // ==========================================================
        // Memory access instruction tests
        // ==========================================================

        // lw x1, 8(x2)
        instr = 32'b000000001000_00010_010_00001_0000011; Zero = 0;
        #10;

        // sw x3, 8(x2)
        instr = 32'b0000000_00011_00010_010_01000_0100011; Zero = 0;
        #10;

        // ==========================================================
        // Branch instruction tests
        // ==========================================================

        // beq taken
        instr = 32'b0000000_00011_00010_000_01000_1100011; Zero = 1;
        #10;

        // beq not taken
        instr = 32'b0000000_00011_00010_000_01000_1100011; Zero = 0;
        #10;

        // bne taken
        instr = 32'b0000000_00011_00010_001_01000_1100011; Zero = 0;
        #10;

        // bne not taken
        instr = 32'b0000000_00011_00010_001_01000_1100011; Zero = 1;
        #10;

        // blt taken (using slt + ~Zero)
        instr = 32'b0000000_00011_00010_100_01000_1100011; Zero = 0;
        #10;
```

```

        // blt not taken
        instr = 32'b0000000_00011_00010_100_01000_1100011; Zero = 1;
        #10;

        // bge taken (using slt + Zero)
        instr = 32'b0000000_00011_00010_101_01000_1100011; Zero = 1;
        #10;

        // bge not taken
        instr = 32'b0000000_00011_00010_101_01000_1100011; Zero = 0;
        #10;

        // ==========================================================
        // Jump instruction tests
        // ==========================================================

        // jal x1, offset
        instr = 32'b0_0000001010_1_00000000_00001_1101111; Zero = 0;
        #10;

        // jalr x1, x2, 4
        instr = 32'b000000000100_00010_000_00001_1100111; Zero = 0;
        #10;

        // ==========================================================
        // U-type instruction test
        // ==========================================================

        // lui x1, 0x12345
        instr = 32'b00010010001101000101_00001_0110111; Zero = 0;
        #10;

        $finish;
    end

endmodule
```

Listing 12. Terminal result of the Control Unit module

```
VCD info: dumpfile control_unit_tb.vcd opened for output.
Time    instr           Zero    RegWrite ImmSrc ALUSrc ALUCtrl MemWrite ResultSrc PCSrc
0   003100b3    0   1    000       0       00010       0        01      00
10000   403100b3    0   1    000       0       01010       0        01      00
20000   003160b3    0   1    000       0       00111       0        01      00
30000   003170b3    0   1    000       0       00011       0        01      00
40000   003110b3    0   1    000       0       00000       0        01      00
50000   003150b3    0   1    000       0       10000       0        01      00
60000   003120b3    0   1    000       0       01001       0        01      00
70000   00510093    0   1    000       1       00010       0        01      00
80000   00516093    0   1    000       1       00111       0        01      00
90000   00517093    0   1    000       1       00011       0        01      00
```

100000	00311093	0	1	000	1	00000	0	01	00
110000	00315093	0	1	000	1	10000	0	01	00
120000	00512093	0	1	000	1	01001	0	01	00
130000	00812083	0	1	000	1	00010	0	10	00
140000	00312423	0	0	001	1	00010	1	00	00
150000	00310463	1	0	010	0	01010	0	00	01
160000	00310463	0	0	010	0	01010	0	00	00
170000	00311463	0	0	010	0	01010	0	00	01
180000	00311463	1	0	010	0	01010	0	00	00
190000	00314463	0	0	010	0	01001	0	00	01
200000	00314463	1	0	010	0	01001	0	00	00
210000	00315463	1	0	010	0	01001	0	00	01
220000	00315463	0	0	010	0	01001	0	00	00
230000	015000ef	0	1	100	0	00000	0	11	01
240000	004100e7	0	1	000	1	00010	0	11	10
250000	123450b7	0	1	011	0	00000	0	00	00

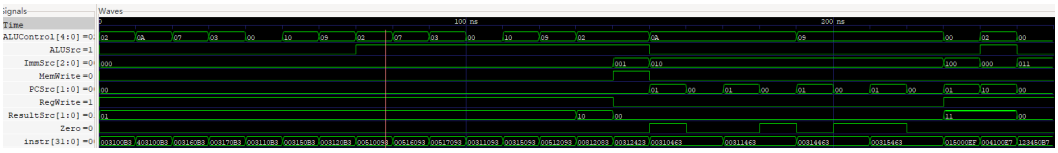


Fig. 7. GTKWave result screenshot for the Control Unit module.

The analysis of the Control Unit testbench is shown below.

The testbench results show that the Control Unit generates the correct control signals for all required RV32I instructions. The outputs for RegWrite, ImmSrc, ALUSrc, ALUControl, MemWrite, ResultSrc, and PCSrc are all consistent with the designed control table.

In particular, branch instructions are handled correctly through the interaction between the Zero flag and the PCSrc selection logic. The BLT and BGE instructions also operate properly by using the SLT-based comparison together with Zero. Although some don't-care fields are assigned fixed values during simulation, they do not affect the functionality of the processor. Overall, the Control Unit meets the coursework requirements and behaves correctly in the tested scenarios.

4 Task1.4

4.1 riscv code

The SystemVerilog implementation of the top-level RISC-V processor is shown below.

Listing 13. Top-level RISC-V module

```
`include "./SVcode/instruction_memory.sv"
`include "./SVcode/reg_file.sv"
`include "./SVcode/data_memory_and_io.sv"
`include "./SVcode/pc/pc.sv"
`include "./SVcode/extend/extend.sv"
`include "./SVcode/alu/alu.sv"
`include "./SVcode/control_unit/control_unit.sv"

module risc_v (
    input  logic        CLK,
```

```
    input  logic        reset,
    input  logic [31:0] CPUIn,
    output logic [31:0] CPUOut
);

    logic [31:0] PC;
    logic [31:0] PCNext;
    logic [31:0] PCPlus4;
    logic [31:0] PCTarget;
    logic [31:0] Instr;
    logic [31:0] RD1;
    logic [31:0] RD2;
    logic [31:0] ImmExt;
    logic [31:0] SrcB;
    logic [31:0] ALUResult;
    logic [31:0] ReadData;
    logic [31:0] Result;
    logic        Zero;

    logic        RegWrite;
    logic [2:0]  ImmSrc;
    logic        ALUSrc;
    logic [4:0]  ALUControl;
    logic        MemWrite;
    logic [1:0]  ResultSrc;
    logic [1:0]  PCSrc;

    assign PCPlus4  = PC + 32'd4;
    assign PCTarget = PC + ImmExt;
    assign SrcB     = ALUSrc ? ImmExt : RD2;

    always_comb begin
        case (ResultSrc)
            2'b00:   Result = ImmExt;
            2'b01:   Result = ALUResult;
            2'b10:   Result = ReadData;
            2'b11:   Result = PCPlus4;
            default: Result = 32'h00000000;
        endcase
    end

    always_comb begin
        case (PCSrc)
            2'b00:   PCNext = PCPlus4;
            2'b01:   PCNext = PCTarget;
            2'b10:   PCNext = ALUResult;
            default: PCNext = PCPlus4;
        endcase
    end

    program_counter u_pc (
        .CLK(CLK),
```

```
        .reset(reset),
        .PCNext(PCNext),
        .PC(PC)
    );

    instruction_memory u_instruction_memory (
        .Instr(Instr),
        .PC(PC)
    );

    control_unit u_control_unit (
        .instr(Instr),
        .Zero(Zero),
        .RegWrite(RegWrite),
        .ImmSrc(ImmSrc),
        .ALUSrc(ALUSrc),
        .ALUControl(ALUControl),
        .MemWrite(MemWrite),
        .ResultSrc(ResultSrc),
        .PCSrc(PCSrc)
    );

    reg_file u_reg_file (
        .RD1(RD1),
        .RD2(RD2),
        .WD3(Result),
        .A1(Instr[19:15]),
        .A2(Instr[24:20]),
        .A3(Instr[11:7]),
        .WE3(RegWrite),
        .CLK(CLK)
    );

    extend u_extend (
        .Instr(Instr),
        .ImmSrc(ImmSrc),
        .ImmExt(ImmExt)
    );

    alu u_alu (
        .A(RD1),
        .B(SrcB),
        .ALUControl(ALUControl),
        .ALUResult(ALUResult),
        .Zero(Zero)
    );

    data_memory_and_io u_data_memory_and_io (
        .RD(ReadData),
        .CPUOut(CPUOut),
        .A(ALUResult),
        .WD(RD2),
```

```
        .CPUIn(CPUIn),
        .WE(MemWrite),
        .CLK(CLK)
    );

endmodule
```

4.2 program description and converting

4.2.1 Disassembly of `program`. The machine code stored in `program.txt` can be disassembled into the following RISC-V assembly instructions:

0x00000000:	03200093	addi x1, x0, 50
0x00000004:	80000537	lui x10, 0x80000
0x00000008:	ffc52103	lw x2, -4(x10)
0x0000000C:	402081b3	sub x3, x1, x2
0x00000010:	fe352e23	sw x3, -4(x10)
0x00000014:	00000063	beq x0, x0, 0

4.2.2 Method for Decoding Machine Code. Each instruction is 32 bits wide. The lowest 7 bits (`instr[6:0]`) define the opcode, which determines the instruction format. Once the format is identified, the remaining fields are extracted from fixed bit positions.

The relevant field mappings are summarised below:

R-type.

- opcode = instr[6:0]
- rd = instr[11:7]
- funct3 = instr[14:12]
- rs1 = instr[19:15]
- rs2 = instr[24:20]
- funct7 = instr[31:25]

I-type.

- opcode = instr[6:0]
- rd = instr[11:7]
- funct3 = instr[14:12]
- rs1 = instr[19:15]
- imm[11:0] = instr[31:20]

S-type.

- opcode = instr[6:0]
- imm[4:0] = instr[11:7]
- funct3 = instr[14:12]
- rs1 = instr[19:15]
- rs2 = instr[24:20]
- imm[11:5] = instr[31:25]

B-type.

- opcode = instr[6:0]
- imm = sign-extended {instr[31], instr[7], instr[30:25], instr[11:8], 1'b0}

U-type.

- opcode = instr[6:0]
- rd = instr[11:7]
- imm[31:12] = instr[31:12]

4.2.3 Step-by-Step Decoding. Each instruction is decoded as follows:

Instruction 1: 03200093. Opcode `0010011` indicates an I-type arithmetic instruction. `funct3 = 000` corresponds to `addi`. Thus, the instruction is:

```
addi x1, x0, 50
```

Instruction 2: 80000537. Opcode `0110111` indicates a U-type instruction (`lui`). The immediate value is shifted left by 12 bits:

$$\texttt{lui x10, 0x80000} \Rightarrow x10 = 0x80000000$$

Instruction 3: ffc52103. Opcode `0000011` indicates a load instruction. `funct3 = 010` selects `lw`. The immediate `0xFFC` is sign-extended to −4, giving:

```
lw x2, -4(x10)
```

Instruction 4: 402081b3. Opcode `0110011` indicates an R-type instruction. `funct7 = 0100000` selects subtraction:

```
sub x3, x1, x2
```

Instruction 5: fe352e23. Opcode `0100011` indicates an S-type instruction. The immediate is reconstructed and sign-extended to −4:

```
sw x3, -4(x10)
```

Instruction 6: 00000063. Opcode `1100011` indicates a branch instruction. `funct3 = 000` corresponds to `beq`. Since both operands are `x0`, the condition is always true:

```
beq x0, x0, 0
```

This results in an infinite loop.

4.2.4 Expected Behaviour. The program constructs the address:

$$0x80000000 - 4 = 0x7FFFFFFC$$

This corresponds to the memory-mapped I/O address used by the `data_memory_and_io` module. The program performs the following steps:

(1) Load the constant 50 into `x1`
(2) Read `CPUIn` from address `0x7FFFFFFC` into `x2`
(3) Compute `x3 = 50 - CPUIn`
(4) Write `x3` to `CPUOut` via a store instruction
(5) Enter an infinite loop using `beq x0, x0, 0`

For the default testbench input `CPUIn = 20`, the expected output is:

```
CPUOut = 30
```

4.3 riscv testbench code

The testbench code of the top-level RISC-V processor is shown below.

Listing 14. RISC-V testbench

```
`timescale 1ns/1ps
`include "./SVcode/risc_v/risc_v.sv"

module risc_v_tb;

    logic        CLK;
    logic        reset;
    logic [31:0] CPUIn;
    logic [31:0] CPUOut;

    risc_v dut (
        .CLK(CLK),
        .reset(reset),
        .CPUIn(CPUIn),
        .CPUOut(CPUOut)
    );

    initial begin
        CLK = 1'b0;
        forever #5 CLK = ~CLK;
    end

    initial begin
        $dumpfile("risc_v_tb.vcd");
        $dumpvars(0, risc_v_tb);

        $dumpvars(0, risc_v_tb.dut.u_reg_file.RF[1]);
        $dumpvars(0, risc_v_tb.dut.u_reg_file.RF[2]);
        $dumpvars(0, risc_v_tb.dut.u_reg_file.RF[3]);

        reset = 1'b1;
        CPUIn = 32'd20;

        #12;
        reset = 1'b0;

        #2000;
        $finish;
    end

    initial begin
        $monitor(
            "t=%0t reset=%b PC=%h Instr=%h CPUIn=%h CPUOut=%h Result=%h ALUResult=%h x1=%h x2=%h x3=%h",
            $time,
            reset,
            dut.PC,
```

```
            dut.Instr,
            CPUIn,
            CPUOut,
            dut.Result,
            dut.ALUResult,
            dut.u_reg_file.RF[1],
            dut.u_reg_file.RF[2],
            dut.u_reg_file.RF[3]
        );
    end

endmodule
```

4.4 terminal result screenshot

Listing 15. Terminal result of the top-level RISC-V simulation

```
VCD info: dumpfile risc_v_tb.vcd opened for output.
t=0 reset=1 PC=xxxxxxxx Instr=xxxxxxxx CPUIn=00000014 CPUOut=xxxxxxxx Result=xxxxxxxx ALUResult
     =xxxxxxxx
t=5000 reset=1 PC=00000000 Instr=80000fb7 CPUIn=00000014 CPUOut=xxxxxxxx Result=80000000
     ALUResult=00000000
t=12000 reset=0 PC=00000000 Instr=80000fb7 CPUIn=00000014 CPUOut=xxxxxxxx Result=80000000
     ALUResult=00000000
t=15000 reset=0 PC=00000004 Instr=ffcf8f93 CPUIn=00000014 CPUOut=xxxxxxxx Result=7ffffffc
     ALUResult=7ffffffc
t=25000 reset=0 PC=00000008 Instr=08000093 CPUIn=00000014 CPUOut=xxxxxxxx Result=00000080
     ALUResult=00000080
t=35000 reset=0 PC=0000000c Instr=001fa023 CPUIn=00000014 CPUOut=xxxxxxxx Result=00000000
     ALUResult=7ffffffc
t=45000 reset=0 PC=00000010 Instr=00100093 CPUIn=00000014 CPUOut=00000080 Result=00000001
     ALUResult=00000001
t=55000 reset=0 PC=00000014 Instr=00300113 CPUIn=00000014 CPUOut=00000080 Result=00000003
     ALUResult=00000003
t=65000 reset=0 PC=00000018 Instr=002081b3 CPUIn=00000014 CPUOut=00000080 Result=00000004
     ALUResult=00000004
t=75000 reset=0 PC=0000001c Instr=003fa023 CPUIn=00000014 CPUOut=00000080 Result=00000000
     ALUResult=7ffffffc
t=85000 reset=0 PC=00000020 Instr=402081b3 CPUIn=00000014 CPUOut=00000004 Result=fffffffe
     ALUResult=fffffffe
t=95000 reset=0 PC=00000024 Instr=003fa023 CPUIn=00000014 CPUOut=00000004 Result=00000000
     ALUResult=7ffffffc
t=105000 reset=0 PC=00000028 Instr=0020e1b3 CPUIn=00000014 CPUOut=fffffffe Result=00000003
     ALUResult=00000003
```

4.5 gtkwave result screenshot

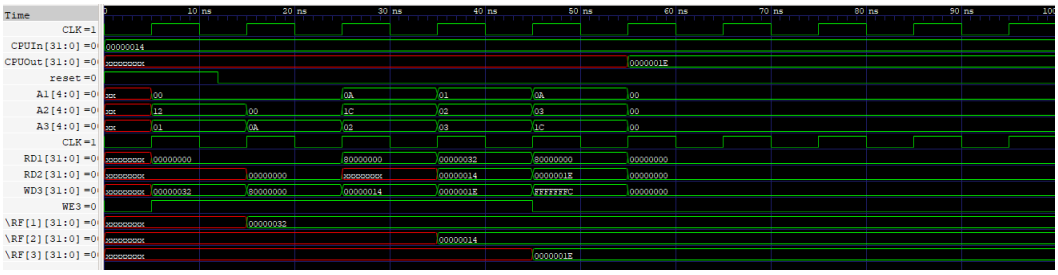


Fig. 8. GTKWave result screenshot for the top-level RISC-V simulation.

4.6 result analyze

The simulation result is correct. The processor successfully executes the program in program.txt, reads the memory-mapped input value CPUIn = 20 from address 0x7FFFFFFC, computes 50 - 20 = 30, and writes the result 0x0000001E to CPUOut. The final beq x0, x0, 0 instruction then causes the processor to loop indefinitely at the same instruction address.

5 Task1.5

5.1 Introduction to the Test Program

This program is a comprehensive test routine designed to verify the correct functionality of a single-cycle RISC-V processor implementation. It begins by constructing the memory-mapped I/O address (`0x7FFFFFFC`) and storing it in register `x31`, which is used to output results through store instructions.

The program then sequentially tests a range of arithmetic, logical, and shift operations, including `addi`, `add`, `sub`, `or`, `sll`, and `slt`, by performing computations on predefined register values and writing the results to the output port.

Next, it evaluates immediate-type instructions such as `andi`, `srli`, and `slti`, ensuring that immediate decoding and sign extension are correctly implemented. Memory access functionality is also verified using `sw` and `lw` instructions, confirming that data can be stored to and retrieved from memory correctly.

The program further tests branch instructions (`beq`, `bne`, `blt`, `bge`) by checking both taken and not-taken conditions, ensuring proper control flow. Finally, it validates jump instructions (`jal` and `jalr`), including correct link register updates and indirect jumps.

Overall, the program systematically exercises all supported instructions and verifies both datapath and control logic correctness.

5.2 Terminal Result Screenshot of the Self-Designed RISC-V Processor

Listing 16. Terminal result of the self-designed RISC-V processor running the test program

```
t=0 reset=1 PC=xxxxxxxx Instr=xxxxxxxx CPUIn=00000014 CPUOut=xxxxxxxx Result=xxxxxxxx ALUResult
     =xxxxxxxx x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=5000 reset=1 PC=00000000 Instr=80000fb7 CPUIn=00000014 CPUOut=xxxxxxxx Result=80000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=12000 reset=0 PC=00000000 Instr=80000fb7 CPUIn=00000014 CPUOut=xxxxxxxx Result=80000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
```

```
t=15000 reset=0 PC=00000004 Instr=ffcf8f93 CPUIn=00000014 CPUOut=xxxxxxxx Result=7fffffc
     ALUResult=7fffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=25000 reset=0 PC=00000008 Instr=08000093 CPUIn=00000014 CPUOut=xxxxxxxx Result=00000080
     ALUResult=00000080 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=35000 reset=0 PC=0000000c Instr=001fa023 CPUIn=00000014 CPUOut=xxxxxxxx Result=00000000
     ALUResult=7fffffc x1=00000080 x2=xxxxxxxx x3=xxxxxxxx
t=45000 reset=0 PC=00000010 Instr=00100093 CPUIn=00000014 CPUOut=00000080 Result=00000001
     ALUResult=00000001 x1=00000080 x2=xxxxxxxx x3=xxxxxxxx
t=55000 reset=0 PC=00000014 Instr=00300113 CPUIn=00000014 CPUOut=00000080 Result=00000003
     ALUResult=00000003 x1=00000001 x2=xxxxxxxx x3=xxxxxxxx
t=65000 reset=0 PC=00000018 Instr=002081b3 CPUIn=00000014 CPUOut=00000080 Result=00000004
     ALUResult=00000004 x1=00000001 x2=00000003 x3=xxxxxxxx
t=75000 reset=0 PC=0000001c Instr=003fa023 CPUIn=00000014 CPUOut=00000080 Result=00000000
     ALUResult=7fffffc x1=00000001 x2=00000003 x3=00000004
t=85000 reset=0 PC=00000020 Instr=402081b3 CPUIn=00000014 CPUOut=00000004 Result=fffffffe
     ALUResult=fffffffe x1=00000001 x2=00000003 x3=00000004
t=95000 reset=0 PC=00000024 Instr=003fa023 CPUIn=00000014 CPUOut=00000004 Result=00000000
     ALUResult=7fffffc x1=00000001 x2=00000003 x3=fffffffe
t=105000 reset=0 PC=00000028 Instr=0020e1b3 CPUIn=00000014 CPUOut=fffffffe Result=00000003
     ALUResult=00000003 x1=00000001 x2=00000003 x3=fffffffe
t=115000 reset=0 PC=0000002c Instr=003fa023 CPUIn=00000014 CPUOut=fffffffe Result=00000000
     ALUResult=7fffffc x1=00000001 x2=00000003 x3=00000003
t=125000 reset=0 PC=00000030 Instr=002091b3 CPUIn=00000014 CPUOut=00000003 Result=00000008
     ALUResult=00000008 x1=00000001 x2=00000003 x3=00000003
t=135000 reset=0 PC=00000034 Instr=003fa023 CPUIn=00000014 CPUOut=00000003 Result=00000000
     ALUResult=7fffffc x1=00000001 x2=00000003 x3=00000008
t=145000 reset=0 PC=00000038 Instr=0020a1b3 CPUIn=00000014 CPUOut=00000008 Result=00000001
     ALUResult=00000001 x1=00000001 x2=00000003 x3=00000008
t=155000 reset=0 PC=0000003c Instr=003fa023 CPUIn=00000014 CPUOut=00000008 Result=00000000
     ALUResult=7fffffc x1=00000001 x2=00000003 x3=00000001
t=165000 reset=0 PC=00000040 Instr=00717193 CPUIn=00000014 CPUOut=00000001 Result=00000003
     ALUResult=00000003 x1=00000001 x2=00000003 x3=00000001
t=175000 reset=0 PC=00000044 Instr=003fa023 CPUIn=00000014 CPUOut=00000001 Result=00000000
     ALUResult=7fffffc x1=00000001 x2=00000003 x3=00000003
t=185000 reset=0 PC=00000048 Instr=00115193 CPUIn=00000014 CPUOut=00000003 Result=00000001
     ALUResult=00000001 x1=00000001 x2=00000003 x3=00000003
t=195000 reset=0 PC=0000004c Instr=003fa023 CPUIn=00000014 CPUOut=00000003 Result=00000000
     ALUResult=7fffffc x1=00000001 x2=00000003 x3=00000001
t=205000 reset=0 PC=00000050 Instr=fff0a193 CPUIn=00000014 CPUOut=00000001 Result=00000000
     ALUResult=00000000 x1=00000001 x2=00000003 x3=00000001
t=215000 reset=0 PC=00000054 Instr=003fa023 CPUIn=00000014 CPUOut=00000001 Result=00000000
     ALUResult=7fffffc x1=00000001 x2=00000003 x3=00000000
t=225000 reset=0 PC=00000058 Instr=00202023 CPUIn=00000014 CPUOut=00000000 Result=00000000
     ALUResult=00000000 x1=00000001 x2=00000003 x3=00000000
t=235000 reset=0 PC=0000005c Instr=00002183 CPUIn=00000014 CPUOut=00000000 Result=00000003
     ALUResult=00000000 x1=00000001 x2=00000003 x3=00000000
t=245000 reset=0 PC=00000060 Instr=003fa023 CPUIn=00000014 CPUOut=00000000 Result=00000000
     ALUResult=7fffffc x1=00000001 x2=00000003 x3=00000003
t=255000 reset=0 PC=00000064 Instr=00208063 CPUIn=00000014 CPUOut=00000003 Result=00000000
     ALUResult=fffffffe x1=00000001 x2=00000003 x3=00000003
t=265000 reset=0 PC=00000068 Instr=00209663 CPUIn=00000014 CPUOut=00000003 Result=0000000c
     ALUResult=fffffffe x1=00000001 x2=00000003 x3=00000003
t=275000 reset=0 PC=00000074 Instr=00114063 CPUIn=00000014 CPUOut=00000003 Result=00000000
     ALUResult=00000000 x1=00000001 x2=00000003 x3=00000003
t=285000 reset=0 PC=00000078 Instr=00115663 CPUIn=00000014 CPUOut=00000003 Result=0000000c
     ALUResult=00000000 x1=00000001 x2=00000003 x3=00000003
```

```
t=295000 reset=0 PC=00000084 Instr=0100026f CPUIn=00000014 CPUOut=00000003 Result=00000088
     ALUResult=xxxxxxxx x1=00000001 x2=00000003 x3=00000003
t=305000 reset=0 PC=00000094 Instr=11100193 CPUIn=00000014 CPUOut=00000003 Result=00000111
     ALUResult=00000111 x1=00000001 x2=00000003 x3=00000003
t=315000 reset=0 PC=00000098 Instr=003fa023 CPUIn=00000014 CPUOut=00000003 Result=00000000
     ALUResult=7ffffffc x1=00000001 x2=00000003 x3=00000111
t=325000 reset=0 PC=0000009c Instr=00020067 CPUIn=00000014 CPUOut=00000111 Result=000000a0
     ALUResult=00000088 x1=00000001 x2=00000003 x3=00000111
t=335000 reset=0 PC=00000088 Instr=78900193 CPUIn=00000014 CPUOut=00000111 Result=00000789
     ALUResult=00000789 x1=00000001 x2=00000003 x3=00000111
t=345000 reset=0 PC=0000008c Instr=003fa023 CPUIn=00000014 CPUOut=00000111 Result=00000000
     ALUResult=7ffffffc x1=00000001 x2=00000003 x3=00000789
t=355000 reset=0 PC=00000090 Instr=0100006f CPUIn=00000014 CPUOut=00000789 Result=00000094
     ALUResult=xxxxxxxx x1=00000001 x2=00000003 x3=00000789
t=365000 reset=0 PC=000000a0 Instr=xxxxxxxx CPUIn=00000014 CPUOut=00000789 Result=xxxxxxxx
     ALUResult=xxxxxxxx x1=00000001 x2=00000003 x3=00000789
t=375000 reset=0 PC=000000a4 Instr=xxxxxxxx CPUIn=00000014 CPUOut=00000789 Result=xxxxxxxx
     ALUResult=xxxxxxxx x1=00000001 x2=00000003 x3=00000789
t=385000 reset=0 PC=000000a8 Instr=xxxxxxxx CPUIn=00000014 CPUOut=00000789 Result=xxxxxxxx
     ALUResult=xxxxxxxx x1=00000001 x2=00000003 x3=00000789
t=395000 reset=0 PC=000000ac Instr=xxxxxxxx CPUIn=00000014 CPUOut=00000789 Result=xxxxxxxx
     ALUResult=xxxxxxxx x1=00000001 x2=00000003 x3=00000789
t=405000 reset=0 PC=000000b0 Instr=xxxxxxxx CPUIn=00000014 CPUOut=00000789 Result=xxxxxxxx
     ALUResult=xxxxxxxx x1=00000001 x2=00000003 x3=00000789
t=415000 reset=0 PC=000000b4 Instr=xxxxxxxx CPUIn=00000014 CPUOut=00000789 Result=xxxxxxxx
     ALUResult=xxxxxxxx x1=00000001 x2=00000003 x3=00000789
t=425000 reset=0 PC=000000b8 Instr=xxxxxxxx CPUIn=00000014 CPUOut=00000789 Result=xxxxxxxx
     ALUResult=xxxxxxxx x1=00000001 x2=00000003 x3=00000789
t=435000 reset=0 PC=000000bc Instr=xxxxxxxx CPUIn=00000014 CPUOut=00000789 Result=xxxxxxxx
     ALUResult=xxxxxxxx x1=00000001 x2=00000003 x3=00000789
```

5.3 GTKWave Result Screenshot of the Self-Designed RISC-V Processor

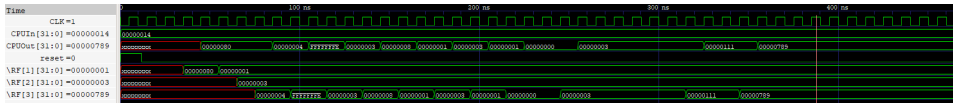
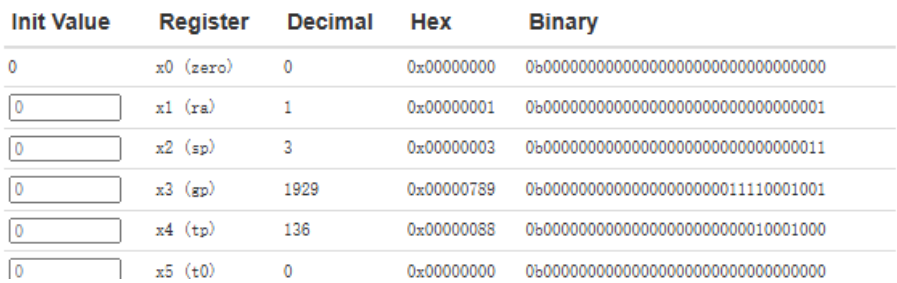


Fig. 9. GTKWave result screenshot for the self-designed RISC-V processor running the test program.

5.4 Results from the Online RISC-V Simulator



Init Value	Register	Decimal	Hex	Binary
0	x0 (zero)	0	0x00000000	0b00000000000000000000000000000000
0	x1 (ra)	1	0x00000001	0b00000000000000000000000000000001
0	x2 (sp)	3	0x00000003	0b00000000000000000000000000000011
0	x3 (gp)	1929	0x00000789	0b00000000000000000000011110001001
0	x4 (tp)	136	0x00000088	0b00000000000000000000000010001000
0	x5 (t0)	0	0x00000000	0b00000000000000000000000000000000

Fig. 10. Online RISC-V simulator result screenshot 1.

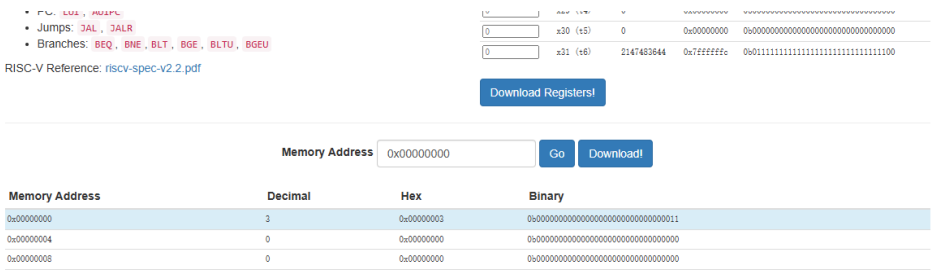


Fig. 11. Online RISC-V simulator result screenshot 2.

5.5 Result Analysis

The simulation results from the implemented RISC-V processor are consistent with the expected behaviour and match those obtained from the reference simulator. The output sequence observed on `CPUOut` correctly reflects the execution of each instruction in the test program. Arithmetic and logical operations such as `add`, `sub`, `or`, `sll`, and `slt` produce the expected results, confirming correct ALU functionality.

Immediate instructions including `andi`, `srli`, and `slti` also behave correctly, indicating proper immediate decoding and sign extension. Memory operations are validated by storing and reloading values, demonstrating correct data memory access. Branch instructions correctly control program flow, as instructions in non-taken paths are skipped. Finally, the correct output value `0x00000789` confirms that both `jal` and `jalr` perform accurate control transfer and link register updates.

Overall, the results verify that both the datapath and control unit operate correctly across all tested instructions.

6 Task2.1

6.1 C code

Listing 17. C reference code for Task 2.1

```
int a = CPUIn;
int sum = 0;
```

```
int i = 0;

while (1) {
    if ((a & 1U) == 1U) {
        sum += 1;
    }

    a = a >> 1;
    i += 1;

    if (i == 8) {
        break;
    }
}

return sum;
```

6.2 RISC-V code

Listing 18. RISC-V code for Task 2.1

```
lui   x5, 0x80000      # x5 = 0x80000000
addi  x5, x5, -4       # x5 = 0x7FFFFFFC (I/O address)

lw    x6, 0(x5)        # a = CPUIn
addi  x7, x0, 0        # sum = 0
addi  x28, x0, 0       # i = 0
addi  x30, x0, 8       # loop limit = 8

loop:
andi  x29, x6, 1       # tmp = a & 1
beq   x29, x0, skip    # if lowest bit == 0, skip increment
addi  x7, x7, 1        # sum += 1

skip:
srli  x6, x6, 1        # a = a >> 1
addi  x28, x28, 1      # i += 1
beq   x28, x30, done   # if i == 8, finish
beq   x0, x0, loop     # unconditional jump back to loop

done:
sw    x7, 0(x5)        # CPUOut = sum
beq   x0, x0, done     # stop here forever
```

Translation from C to RISC-V Assembly. This program was translated from C to RISC-V assembly by first identifying the variables and the control flow in the original C implementation. The variable `a` is stored in a register and loaded from `CPUIn` through the memory-mapped I/O address `0x7FFFFFFC`. The variables `sum` and `i` are also assigned to registers and are initialised to zero using `addi`.

The `while` loop in C is translated into a loop label in assembly. The expression `(a & 1U)` is implemented using `andi`, because this instruction directly extracts the least significant bit. The `if` statement is implemented using `beq`: if the extracted bit is zero, the program branches to `skip`; otherwise, it executes `addi` to increment `sum`. The statements `a = a » 1` and `i += 1` are implemented using `srli` and `addi`, respectively. The termination condition `i == 8` is checked using `beq`, and if the condition is true, the program branches to `done`. Finally, `sum` is written to `CPUOut` using `sw`.

6.3 Machine code

Listing 19. Machine code for Task 2.1

```
800002b7
ffc28293
0002a303
00000393
00000e13
00800f13
00137e93
000e8463
00138393
00135313
001e0e13
01ee0463
fe0004e3
0072a023
fe000ee3
```

6.4 testbench

Listing 20. Testbench for Task 2.1

```
`timescale 1ns/1ps
`include "./SVcode/risc_v/risc_v.sv"

module risc_v_tb;

    logic        CLK;
    logic        reset;
    logic [31:0] CPUIn;
    logic [31:0] CPUOut;

    risc_v dut (
        .CLK(CLK),
        .reset(reset),
        .CPUIn(CPUIn),
        .CPUOut(CPUOut)
    );

    initial begin
        CLK = 1'b0;
```

```
        forever #5 CLK = ~CLK;
    end

    initial begin
        $dumpfile("risc_v_tb.vcd");
        $dumpvars(0, risc_v_tb);

        $dumpvars(0, risc_v_tb.dut.u_reg_file.RF[1]);
        $dumpvars(0, risc_v_tb.dut.u_reg_file.RF[2]);
        $dumpvars(0, risc_v_tb.dut.u_reg_file.RF[3]);

        reset = 1'b1;
        CPUIn = 32'b0;

        #12;
        reset = 1'b0;

        #1000;
        $finish;
    end

    initial begin
        $monitor(
            "t=%0t reset=%b PC=%h Instr=%h CPUIn=%h CPUOut=%h Result=%h ALUResult=%h x1=%h x2=%h x3=%h",
            $time,
            reset,
            dut.PC,
            dut.Instr,
            CPUIn,
            CPUOut,
            dut.Result,
            dut.ALUResult,
            dut.u_reg_file.RF[1],
            dut.u_reg_file.RF[2],
            dut.u_reg_file.RF[3]
        );
    end

endmodule
```

6.5 result 1

Listing 21. Terminal result 1 for Task 2.1

```
t=0 reset=1 PC=xxxxxxxx Instr=xxxxxxxx CPUIn=00004959 CPUOut=xxxxxxxx Result=xxxxxxxx ALUResult=xxxxxxxx x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=5000 reset=1 PC=00000000 Instr=800002b7 CPUIn=00004959 CPUOut=xxxxxxxx Result=80000000 ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=12000 reset=0 PC=00000000 Instr=800002b7 CPUIn=00004959 CPUOut=xxxxxxxx Result=80000000 ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
```

```
t=15000 reset=0 PC=00000004 Instr=ffc28293 CPUIn=00004959 CPUOut=xxxxxxxx Result=7ffffffc
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=25000 reset=0 PC=00000008 Instr=0002a303 CPUIn=00004959 CPUOut=xxxxxxxx Result=00004959
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=35000 reset=0 PC=0000000c Instr=00000393 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=45000 reset=0 PC=00000010 Instr=00000e13 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=55000 reset=0 PC=00000014 Instr=00800f13 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000008 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=65000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=75000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=85000 reset=0 PC=00000020 Instr=00138393 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=95000 reset=0 PC=00000024 Instr=00135313 CPUIn=00004959 CPUOut=xxxxxxxx Result=000024ac
     ALUResult=000024ac x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=105000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=115000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffff9 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=125000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=00004959 CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=135000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=145000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=155000 reset=0 PC=00000024 Instr=00135313 CPUIn=00004959 CPUOut=xxxxxxxx Result=00001256
     ALUResult=00001256 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=165000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000002
     ALUResult=00000002 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=175000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffa x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=185000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=00004959 CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=195000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=205000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=215000 reset=0 PC=00000024 Instr=00135313 CPUIn=00004959 CPUOut=xxxxxxxx Result=0000092b
     ALUResult=0000092b x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=225000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000003
     ALUResult=00000003 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=235000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffb x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=245000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=00004959 CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=255000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=265000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=275000 reset=0 PC=00000020 Instr=00138393 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000002
     ALUResult=00000002 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=285000 reset=0 PC=00000024 Instr=00135313 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000495
     ALUResult=00000495 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
```

```
t=295000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000004
     ALUResult=00000004 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=305000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=315000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=00004959 CPUOut=xxxxxxxx Result=fffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=325000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=335000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=345000 reset=0 PC=00000020 Instr=00138393 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000003
     ALUResult=00000003 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=355000 reset=0 PC=00000024 Instr=00135313 CPUIn=00004959 CPUOut=xxxxxxxx Result=0000024a
     ALUResult=0000024a x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=365000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000005
     ALUResult=00000005 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=375000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffd x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=385000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=00004959 CPUOut=xxxxxxxx Result=fffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=395000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=405000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=415000 reset=0 PC=00000024 Instr=00135313 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000125
     ALUResult=00000125 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=425000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000006
     ALUResult=00000006 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=435000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffe x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=445000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=00004959 CPUOut=xxxxxxxx Result=fffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=455000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=465000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=475000 reset=0 PC=00000020 Instr=00138393 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000004
     ALUResult=00000004 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=485000 reset=0 PC=00000024 Instr=00135313 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000092
     ALUResult=00000092 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=495000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000007
     ALUResult=00000007 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=505000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=ffffffff x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=515000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=00004959 CPUOut=xxxxxxxx Result=fffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=525000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=535000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=545000 reset=0 PC=00000024 Instr=00135313 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000049
     ALUResult=00000049 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=555000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000008 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=565000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
```

```
t=575000 reset=0 PC=00000034 Instr=0072a023 CPUIn=00004959 CPUOut=xxxxxxxx Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=585000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=00004959 CPUOut=00000004 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=595000 reset=0 PC=00000034 Instr=0072a023 CPUIn=00004959 CPUOut=00000004 Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=605000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=00004959 CPUOut=00000004 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=615000 reset=0 PC=00000034 Instr=0072a023 CPUIn=00004959 CPUOut=00000004 Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=625000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=00004959 CPUOut=00000004 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=635000 reset=0 PC=00000034 Instr=0072a023 CPUIn=00004959 CPUOut=00000004 Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=645000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=00004959 CPUOut=00000004 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=655000 reset=0 PC=00000034 Instr=0072a023 CPUIn=00004959 CPUOut=00000004 Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=665000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=00004959 CPUOut=00000004 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=675000 reset=0 PC=00000034 Instr=0072a023 CPUIn=00004959 CPUOut=00000004 Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=685000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=00004959 CPUOut=00000004 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=695000 reset=0 PC=00000034 Instr=0072a023 CPUIn=00004959 CPUOut=00000004 Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=705000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=00004959 CPUOut=00000004 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=715000 reset=0 PC=00000034 Instr=0072a023 CPUIn=00004959 CPUOut=00000004 Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=725000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=00004959 CPUOut=00000004 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=735000 reset=0 PC=00000034 Instr=0072a023 CPUIn=00004959 CPUOut=00000004 Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
```

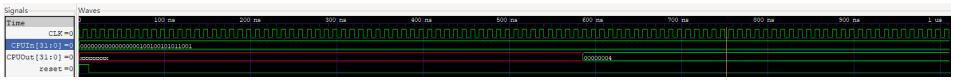


Fig. 12. GTKWave result 1 for Task 2.1.

6.6 result 2

Listing 22. Terminal result 2 for Task 2.1

```
t=0 reset=1 PC=xxxxxxxx Instr=xxxxxxxx CPUIn=000007ff CPUOut=xxxxxxxx Result=xxxxxxxx ALUResult
     =xxxxxxxx x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=5000 reset=1 PC=00000000 Instr=800002b7 CPUIn=000007ff CPUOut=xxxxxxxx Result=80000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=12000 reset=0 PC=00000000 Instr=800002b7 CPUIn=000007ff CPUOut=xxxxxxxx Result=80000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=15000 reset=0 PC=00000004 Instr=ffc28293 CPUIn=000007ff CPUOut=xxxxxxxx Result=7ffffffc
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=25000 reset=0 PC=00000008 Instr=0002a303 CPUIn=000007ff CPUOut=xxxxxxxx Result=000007ff
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
```

```
t=35000 reset=0 PC=0000000c Instr=00000393 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=45000 reset=0 PC=00000010 Instr=00000e13 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=55000 reset=0 PC=00000014 Instr=00800f13 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000008 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=65000 reset=0 PC=00000018 Instr=00137e93 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=75000 reset=0 PC=0000001c Instr=000e8463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=85000 reset=0 PC=00000020 Instr=00138393 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=95000 reset=0 PC=00000024 Instr=00135313 CPUIn=000007ff CPUOut=xxxxxxxx Result=000003ff
     ALUResult=000003ff x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=105000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=115000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffff9 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=125000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=000007ff CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=135000 reset=0 PC=00000018 Instr=00137e93 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=145000 reset=0 PC=0000001c Instr=000e8463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=155000 reset=0 PC=00000020 Instr=00138393 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000002
     ALUResult=00000002 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=165000 reset=0 PC=00000024 Instr=00135313 CPUIn=000007ff CPUOut=xxxxxxxx Result=000001ff
     ALUResult=000001ff x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=175000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000002
     ALUResult=00000002 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=185000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffa x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=195000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=000007ff CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=205000 reset=0 PC=00000018 Instr=00137e93 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=215000 reset=0 PC=0000001c Instr=000e8463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=225000 reset=0 PC=00000020 Instr=00138393 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000003
     ALUResult=00000003 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=235000 reset=0 PC=00000024 Instr=00135313 CPUIn=000007ff CPUOut=xxxxxxxx Result=000000ff
     ALUResult=000000ff x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=245000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000003
     ALUResult=00000003 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=255000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffb x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=265000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=000007ff CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=275000 reset=0 PC=00000018 Instr=00137e93 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=285000 reset=0 PC=0000001c Instr=000e8463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=295000 reset=0 PC=00000020 Instr=00138393 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000004
     ALUResult=00000004 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=305000 reset=0 PC=00000024 Instr=00135313 CPUIn=000007ff CPUOut=xxxxxxxx Result=0000007f
     ALUResult=0000007f x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
```

```
t=315000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000004
     ALUResult=00000004 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=325000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=335000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=000007ff CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=345000 reset=0 PC=00000018 Instr=00137e93 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=355000 reset=0 PC=0000001c Instr=000e8463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=365000 reset=0 PC=00000020 Instr=00138393 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000005
     ALUResult=00000005 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=375000 reset=0 PC=00000024 Instr=00135313 CPUIn=000007ff CPUOut=xxxxxxxx Result=0000003f
     ALUResult=0000003f x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=385000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000005
     ALUResult=00000005 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=395000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffd x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=405000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=000007ff CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=415000 reset=0 PC=00000018 Instr=00137e93 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=425000 reset=0 PC=0000001c Instr=000e8463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=435000 reset=0 PC=00000020 Instr=00138393 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000006
     ALUResult=00000006 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=445000 reset=0 PC=00000024 Instr=00135313 CPUIn=000007ff CPUOut=xxxxxxxx Result=0000001f
     ALUResult=0000001f x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=455000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000006
     ALUResult=00000006 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=465000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffe x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=475000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=000007ff CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=485000 reset=0 PC=00000018 Instr=00137e93 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=495000 reset=0 PC=0000001c Instr=000e8463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=505000 reset=0 PC=00000020 Instr=00138393 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000007
     ALUResult=00000007 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=515000 reset=0 PC=00000024 Instr=00135313 CPUIn=000007ff CPUOut=xxxxxxxx Result=0000000f
     ALUResult=0000000f x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=525000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000007
     ALUResult=00000007 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=535000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=ffffffff x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=545000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=000007ff CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=555000 reset=0 PC=00000018 Instr=00137e93 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=565000 reset=0 PC=0000001c Instr=000e8463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=575000 reset=0 PC=00000020 Instr=00138393 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000008 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=585000 reset=0 PC=00000024 Instr=00135313 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000007
     ALUResult=00000007 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
```

```
t=595000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000008 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=605000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=615000 reset=0 PC=00000034 Instr=0072a023 CPUIn=000007ff CPUOut=xxxxxxxx Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=625000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=000007ff CPUOut=00000008 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=635000 reset=0 PC=00000034 Instr=0072a023 CPUIn=000007ff CPUOut=00000008 Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=645000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=000007ff CPUOut=00000008 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=655000 reset=0 PC=00000034 Instr=0072a023 CPUIn=000007ff CPUOut=00000008 Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=665000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=000007ff CPUOut=00000008 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=675000 reset=0 PC=00000034 Instr=0072a023 CPUIn=000007ff CPUOut=00000008 Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=685000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=000007ff CPUOut=00000008 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
```

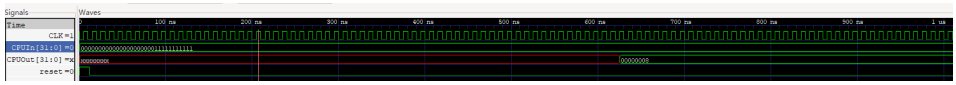


Fig. 13. GTKWave result 2 for Task 2.1.

6.7 result 3

Listing 23. Terminal result 3 for Task 2.1

```
t=0 reset=1 PC=xxxxxxxx Instr=xxxxxxxx CPUIn=00000000 CPUOut=xxxxxxxx Result=xxxxxxxx ALUResult
     =xxxxxxxx x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=5000 reset=1 PC=00000000 Instr=800002b7 CPUIn=00000000 CPUOut=xxxxxxxx Result=80000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=12000 reset=0 PC=00000000 Instr=800002b7 CPUIn=00000000 CPUOut=xxxxxxxx Result=80000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=15000 reset=0 PC=00000004 Instr=ffc28293 CPUIn=00000000 CPUOut=xxxxxxxx Result=7ffffffc
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=25000 reset=0 PC=00000008 Instr=0002a303 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=35000 reset=0 PC=0000000c Instr=00000393 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=45000 reset=0 PC=00000010 Instr=00000e13 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=55000 reset=0 PC=00000014 Instr=00800f13 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000008 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=65000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=75000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=85000 reset=0 PC=00000024 Instr=00135313 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=95000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000001
     ALUResult=00000001 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
```

```
t=105000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffff9 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=115000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=00000000 CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=125000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=135000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=145000 reset=0 PC=00000024 Instr=00135313 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=155000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000002
     ALUResult=00000002 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=165000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffa x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=175000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=00000000 CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=185000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=195000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=205000 reset=0 PC=00000024 Instr=00135313 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=215000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000003
     ALUResult=00000003 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=225000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffb x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=235000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=00000000 CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=245000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=255000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=265000 reset=0 PC=00000024 Instr=00135313 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=275000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000004
     ALUResult=00000004 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=285000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=295000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=00000000 CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=305000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=315000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=325000 reset=0 PC=00000024 Instr=00135313 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=335000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000005
     ALUResult=00000005 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=345000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffd x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=355000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=00000000 CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=365000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=375000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
```

```
t=385000 reset=0 PC=00000024 Instr=00135313 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=395000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000006
     ALUResult=00000006 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=405000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=fffffffe x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=415000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=00000000 CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=425000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=435000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=445000 reset=0 PC=00000024 Instr=00135313 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=455000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000007
     ALUResult=00000007 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=465000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=ffffffff x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=475000 reset=0 PC=00000030 Instr=fe0004e3 CPUIn=00000000 CPUOut=xxxxxxxx Result=ffffffe8
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=485000 reset=0 PC=00000018 Instr=00137e93 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=495000 reset=0 PC=0000001c Instr=000e8463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=505000 reset=0 PC=00000024 Instr=00135313 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=515000 reset=0 PC=00000028 Instr=001e0e13 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000008 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=525000 reset=0 PC=0000002c Instr=01ee0463 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000008
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=535000 reset=0 PC=00000034 Instr=0072a023 CPUIn=00000000 CPUOut=xxxxxxxx Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=545000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=00000000 CPUOut=00000000 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=555000 reset=0 PC=00000034 Instr=0072a023 CPUIn=00000000 CPUOut=00000000 Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=565000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=00000000 CPUOut=00000000 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=575000 reset=0 PC=00000034 Instr=0072a023 CPUIn=00000000 CPUOut=00000000 Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=585000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=00000000 CPUOut=00000000 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=595000 reset=0 PC=00000034 Instr=0072a023 CPUIn=00000000 CPUOut=00000000 Result=00000000
     ALUResult=7ffffffc x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
t=605000 reset=0 PC=00000038 Instr=fe000ee3 CPUIn=00000000 CPUOut=00000000 Result=fffffffc
     ALUResult=00000000 x1=xxxxxxxx x2=xxxxxxxx x3=xxxxxxxx
```

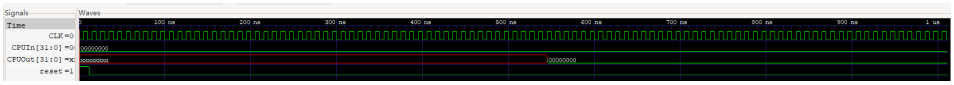


Fig. 14. GTKWave result 3 for Task 2.1.

6.8 analyze

Test Set 1: `output_vcd_1`. The input value is `CPUIn = 0x00004959`. The program only counts the number of 1s in the least significant byte, so the effective input is `0x59`, which is `01011001` in binary. This byte contains four 1s, so the expected result is `CPUOut = 0x00000004`. In the simulation trace, the final stable output is `CPUOut = 00000004`, so this test result is correct.

Test Set 2: `output_vcd_2`. The input value is `CPUIn = 0x000007FF`. The least significant byte is `0xFF`, which is `11111111` in binary. This byte contains eight 1s, so the expected result is `CPUOut = 0x00000008`. In the recorded result, the final output is `CPUOut = 00000008`, so this test result is correct.

Test Set 3: `output_vcd_3`. The input value is `CPUIn = 0x00000000`. The least significant byte is `0x00`, which contains no 1s. Therefore, the expected result is `CPUOut = 0x00000000`. The simulation output ends with `CPUOut = 00000000`, so this test result is correct.

Conclusion. All three Task 2.1 test sets produce the correct output. This shows that the RISC-V program correctly reads the input, repeatedly checks the least significant bit, shifts the input to the right, counts the number of 1s over eight iterations, and writes the final count to `CPUOut`.

7 Task2.2

7.1 C code

Listing 24. C reference code for Task 2.2

```
int P = 0;
int i = 0;

while (1) {
    if ((B & 1) == 1) {
        P = P + A;
    }

    A = A << 1;
    B = B >> 1;
    i = i + 1;

    if (i == 8) {
        break;
    }
}

return P;
```

7.2 RISC-V code

Listing 25. RISC-V code for Task 2.2

```
addi t1, x0, 0
addi s2, x0, 0
addi t3, x0, 8
```

```

loop:
andi t2, s1, 1
beq  t2, x0, skip_add
add  s2, s2, s0

skip_add:
slli s0, s0, 1
srli s1, s1, 1
addi t1, t1, 1
blt  t1, t3, loop

lui  t0, 0x80000
addi t0, t0, -4
sw   s2, 0(t0)
jal  x0, 0
```

Translation from C to RISC-V Assembly. In this task, `A` and `B` are preloaded into registers `s0` and `s1`, respectively. The product `P` is stored in `s2`, and the loop counter `i` is stored in `t1`. The constant `8` is also stored in a register so that the loop termination condition can be checked directly in hardware.

The `while` loop is implemented in assembly using a loop label. The condition `(B & 1) == 1` is realised using `andi` to isolate the least significant bit of `B`, followed by `beq` to skip the addition when this bit is zero. The statement `P = P + A` is implemented using `add`. The shift operations in the C code are translated directly into `slli` and `srli`, while `i = i + 1` becomes `addi`. The condition `i == 8` is implemented by comparing `i` with the loop limit register using `blt`; if `i` is still smaller than `8`, the program branches back to the loop. After the loop finishes, the result `P` is written to `CPUOut` through the memory-mapped output address `0x7FFFFFFC`.

7.3 Machine code

Listing 26. Machine code for Task 2.2

```
00000313
00000913
00800e13
0014f393
00038463
00890933
00141413
0014d493
00130313
ffc344e3
800002b7
ffc28293
0122a023
0000006f
```

7.4 testbench

Listing 27. Testbench for Task 2.2

```
`timescale 1ns/1ps
`include "./SVcode/risc_v/risc_v.sv"
module risc_v_tb;

    logic        CLK;
    logic        reset;
    logic [31:0] CPUIn;
    logic [31:0] CPUOut;

    localparam logic [31:0] A_INIT = 32'd 14;
    localparam logic [31:0] B_INIT = 32'd 15;

    risc_v dut (
        .CLK(CLK),
        .reset(reset),
        .CPUIn(CPUIn),
        .CPUOut(CPUOut)
    );

    initial begin
        CLK = 1'b0;
        forever #5 CLK = ~CLK;
    end

    initial begin
        $dumpfile("risc_v_tb.vcd");
        $dumpvars(0, risc_v_tb);

        $dumpvars(0, risc_v_tb.dut.u_reg_file.RF[8]);
        $dumpvars(0, risc_v_tb.dut.u_reg_file.RF[9]);
        $dumpvars(0, risc_v_tb.dut.u_reg_file.RF[18]);

        reset = 1'b1;
        CPUIn = 32'h00000000;

        dut.u_reg_file.RF[5]  = 32'h00000000;
        dut.u_reg_file.RF[6]  = 32'h00000000;
        dut.u_reg_file.RF[7]  = 32'h00000000;
        dut.u_reg_file.RF[8]  = A_INIT;
        dut.u_reg_file.RF[9]  = B_INIT;
        dut.u_reg_file.RF[18] = 32'h00000000;
        dut.u_reg_file.RF[28] = 32'h00000000;

        #12;
        reset = 1'b0;

        #1500;
        $finish;
    end

```

```
    initial begin
        $monitor(
            "t=%0t reset=%b PC=%h Instr=%h CPUOut=%h A(s0)=%h B(s1)=%h P(s2)=%h i(t1)
    ↪ =%h bit(t2)=%h limit(t3)=%h Result=%h ALUResult=%h",
            $time,
            reset,
            dut.PC,
            dut.Instr,
            CPUOut,
            dut.u_reg_file.RF[8],
            dut.u_reg_file.RF[9],
            dut.u_reg_file.RF[18],
            dut.u_reg_file.RF[6],
            dut.u_reg_file.RF[7],
            dut.u_reg_file.RF[28],
            dut.Result,
            dut.ALUResult
        );
    end

endmodule
```

7.5 result 1

Listing 28. Terminal result 1 for Task 2.2

```
t=0 reset=1 PC=xxxxxxxx Instr=xxxxxxxx CPUOut=xxxxxxxx A(s0)=00000003 B(s1)=00000004 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=xxxxxxxx ALUResult=
     xxxxxxxx
t=5000 reset=1 PC=00000000 Instr=00000313 CPUOut=xxxxxxxx A(s0)=00000003 B(s1)=00000004 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=00000000 ALUResult
     =00000000
t=12000 reset=0 PC=00000000 Instr=00000313 CPUOut=xxxxxxxx A(s0)=00000003 B(s1)=00000004 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=00000000 ALUResult
     =00000000
t=15000 reset=0 PC=00000004 Instr=00000913 CPUOut=xxxxxxxx A(s0)=00000003 B(s1)=00000004 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=00000000 ALUResult
     =00000000
t=25000 reset=0 PC=00000008 Instr=00800e13 CPUOut=xxxxxxxx A(s0)=00000003 B(s1)=00000004 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=00000008 ALUResult
     =00000008
t=35000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000003 B(s1)=00000004 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=45000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000003 B(s1)=00000004 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=55000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000003 B(s1)=00000004 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000008 Result=00000006 ALUResult
     =00000006
t=65000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000006 B(s1)=00000004 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000008 Result=00000002 ALUResult
     =00000002
```

```
t=75000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000006 B(s1)=00000002 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000008 Result=00000001 ALUResult
     =00000001
t=85000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000006 B(s1)=00000002 P(s2)
     =00000000 i(t1)=00000001 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=95000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000006 B(s1)=00000002 P(s2)
     =00000000 i(t1)=00000001 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=105000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000006 B(s1)=00000002 P(s2)
     =00000000 i(t1)=00000001 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=115000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000006 B(s1)=00000002 P(s2)
     =00000000 i(t1)=00000001 bit(t2)=00000000 limit(t3)=00000008 Result=0000000c ALUResult
     =0000000c
t=125000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=0000000c B(s1)=00000002 P(s2)
     =00000000 i(t1)=00000001 bit(t2)=00000000 limit(t3)=00000008 Result=00000001 ALUResult
     =00000001
t=135000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=0000000c B(s1)=00000001 P(s2)
     =00000000 i(t1)=00000001 bit(t2)=00000000 limit(t3)=00000008 Result=00000002 ALUResult
     =00000002
t=145000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=0000000c B(s1)=00000001 P(s2)
     =00000000 i(t1)=00000002 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=155000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=0000000c B(s1)=00000001 P(s2)
     =00000000 i(t1)=00000002 bit(t2)=00000000 limit(t3)=00000008 Result=00000001 ALUResult
     =00000001
t=165000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=0000000c B(s1)=00000001 P(s2)
     =00000000 i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000008 ALUResult
     =00000001
t=175000 reset=0 PC=00000014 Instr=00890933 CPUOut=xxxxxxxx A(s0)=0000000c B(s1)=00000001 P(s2)
     =00000000 i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=0000000c ALUResult
     =0000000c
t=185000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=0000000c B(s1)=00000001 P(s2)
     =0000000c i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000018 ALUResult
     =00000018
t=195000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000018 B(s1)=00000001 P(s2)
     =0000000c i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=205000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000018 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000003 ALUResult
     =00000003
t=215000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000018 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000003 bit(t2)=00000001 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=225000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000018 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000003 bit(t2)=00000001 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=235000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000018 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000003 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=245000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000018 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000003 bit(t2)=00000000 limit(t3)=00000008 Result=00000030 ALUResult
     =00000030
t=255000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000030 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000003 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
```

```
t=265000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000030 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000003 bit(t2)=00000000 limit(t3)=00000008 Result=00000004 ALUResult
     =00000004
t=275000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000030 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=285000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000030 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=295000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000030 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=305000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000030 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=00000060 ALUResult
     =00000060
t=315000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000060 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=325000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000060 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=00000005 ALUResult
     =00000005
t=335000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000060 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=345000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000060 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=355000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000060 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=365000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000060 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=000000c0 ALUResult
     =000000c0
t=375000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=000000c0 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=385000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=000000c0 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=00000006 ALUResult
     =00000006
t=395000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=000000c0 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=405000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=000000c0 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=415000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=000000c0 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=425000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=000000c0 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000180 ALUResult
     =00000180
t=435000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000180 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=445000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000180 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000007 ALUResult
     =00000007
```

```
t=455000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000180 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=465000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000180 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=475000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000180 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=485000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000180 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000300 ALUResult
     =00000300
t=495000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000300 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=505000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000300 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000008
t=515000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000300 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000000
t=525000 reset=0 PC=00000028 Instr=800002b7 CPUOut=xxxxxxxx A(s0)=00000300 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=80000000 ALUResult
     =00000000
t=535000 reset=0 PC=0000002c Instr=ffc28293 CPUOut=xxxxxxxx A(s0)=00000300 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=7ffffffc ALUResult=7
     ffffffc
t=545000 reset=0 PC=00000030 Instr=0122a023 CPUOut=xxxxxxxx A(s0)=00000300 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult=7
     ffffffc
t=555000 reset=0 PC=00000034 Instr=0000006f CPUOut=0000000c A(s0)=00000300 B(s1)=00000000 P(s2)
     =0000000c i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=00000038 ALUResult
     =00000000
```

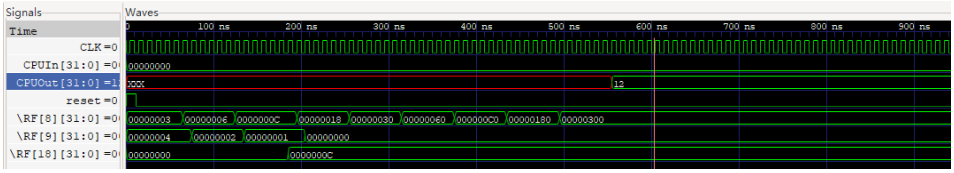


Fig. 15. GTKWave result 1 for Task 2.2.

7.6 result 2

Listing 29. Terminal result 2 for Task 2.2

```
t=0 reset=1 PC=xxxxxxxx Instr=xxxxxxxx CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000005 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=xxxxxxxx ALUResult=
     xxxxxxxx
t=5000 reset=1 PC=00000000 Instr=00000313 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000005 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=00000000 ALUResult
     =00000000
```

```
t=12000 reset=0 PC=00000000 Instr=00000313 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000005 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=00000000 ALUResult
     =00000000
t=15000 reset=0 PC=00000004 Instr=00000913 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000005 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=00000000 ALUResult
     =00000000
t=25000 reset=0 PC=00000008 Instr=00800e13 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000005 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=00000008 ALUResult
     =00000008
t=35000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000005 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000008 Result=00000001 ALUResult
     =00000001
t=45000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000005 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000001 limit(t3)=00000008 Result=00000008 ALUResult
     =00000001
t=55000 reset=0 PC=00000014 Instr=00890933 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000005 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000001 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=65000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000005 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000001 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=75000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000005 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000001 limit(t3)=00000008 Result=00000002 ALUResult
     =00000002
t=85000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000002 P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000001 limit(t3)=00000008 Result=00000001 ALUResult
     =00000001
t=95000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000002 P(s2)
     =00000000 i(t1)=00000001 bit(t2)=00000001 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=105000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000002 P(s2)
     =00000000 i(t1)=00000001 bit(t2)=00000001 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=115000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000002 P(s2)
     =00000000 i(t1)=00000001 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=125000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000002 P(s2)
     =00000000 i(t1)=00000001 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=135000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000002 P(s2)
     =00000000 i(t1)=00000001 bit(t2)=00000000 limit(t3)=00000008 Result=00000001 ALUResult
     =00000001
t=145000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000001 P(s2)
     =00000000 i(t1)=00000001 bit(t2)=00000000 limit(t3)=00000008 Result=00000002 ALUResult
     =00000002
t=155000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000001 P(s2)
     =00000000 i(t1)=00000002 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=165000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000001 P(s2)
     =00000000 i(t1)=00000002 bit(t2)=00000000 limit(t3)=00000008 Result=00000001 ALUResult
     =00000001
t=175000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000001 P(s2)
     =00000000 i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000008 ALUResult
     =00000001
t=185000 reset=0 PC=00000014 Instr=00890933 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000001 P(s2)
     =00000000 i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
```

```
t=195000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000001 P(s2)
     =00000000 i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=205000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000001 P(s2)
     =00000000 i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=215000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000003 ALUResult
     =00000003
t=225000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000003 bit(t2)=00000001 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=235000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000003 bit(t2)=00000001 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=245000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000003 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=255000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000003 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=265000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000003 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=275000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000003 bit(t2)=00000000 limit(t3)=00000008 Result=00000004 ALUResult
     =00000004
t=285000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=295000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=305000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=315000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=325000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=335000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=00000005 ALUResult
     =00000005
t=345000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=355000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=365000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=375000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
```

```
t=385000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=395000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=00000006 ALUResult
     =00000006
t=405000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=415000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=425000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=435000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=445000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=455000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000007 ALUResult
     =00000007
t=465000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=475000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=485000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=495000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=505000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=515000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000008
t=525000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000000
t=535000 reset=0 PC=00000028 Instr=800002b7 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=80000000 ALUResult
     =00000000
t=545000 reset=0 PC=0000002c Instr=ffc28293 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=7ffffffc ALUResult=7
     ffffffc
t=555000 reset=0 PC=00000030 Instr=0122a023 CPUOut=xxxxxxxx A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult=7
     ffffffc
t=565000 reset=0 PC=00000034 Instr=0000006f CPUOut=00000000 A(s0)=00000000 B(s1)=00000000 P(s2)
     =00000000 i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=00000038 ALUResult
     =00000000
```

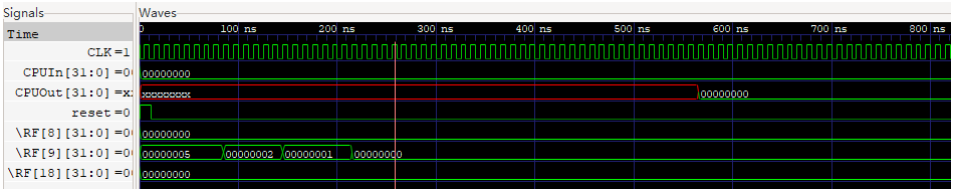


Fig. 16. GTKWave result 2 for Task 2.2.

7.7 result 3

Listing 30. Terminal result 3 for Task 2.2

```
t=0 reset=1 PC=xxxxxxxx Instr=xxxxxxxx CPUOut=xxxxxxxx A(s0)=0000000e B(s1)=0000000f P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=xxxxxxxx ALUResult=
     xxxxxxxx
t=5000 reset=1 PC=00000000 Instr=00000313 CPUOut=xxxxxxxx A(s0)=0000000e B(s1)=0000000f P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=00000000 ALUResult
     =00000000
t=12000 reset=0 PC=00000000 Instr=00000313 CPUOut=xxxxxxxx A(s0)=0000000e B(s1)=0000000f P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=00000000 ALUResult
     =00000000
t=15000 reset=0 PC=00000004 Instr=00000913 CPUOut=xxxxxxxx A(s0)=0000000e B(s1)=0000000f P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=00000000 ALUResult
     =00000000
t=25000 reset=0 PC=00000008 Instr=00800e13 CPUOut=xxxxxxxx A(s0)=0000000e B(s1)=0000000f P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000000 Result=00000008 ALUResult
     =00000008
t=35000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=0000000e B(s1)=0000000f P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000000 limit(t3)=00000008 Result=00000001 ALUResult
     =00000001
t=45000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=0000000e B(s1)=0000000f P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000001 limit(t3)=00000008 Result=00000008 ALUResult
     =00000001
t=55000 reset=0 PC=00000014 Instr=00890933 CPUOut=xxxxxxxx A(s0)=0000000e B(s1)=0000000f P(s2)
     =00000000 i(t1)=00000000 bit(t2)=00000001 limit(t3)=00000008 Result=0000000e ALUResult
     =0000000e
t=65000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=0000000e B(s1)=0000000f P(s2)
     =0000000e i(t1)=00000000 bit(t2)=00000001 limit(t3)=00000008 Result=0000001c ALUResult
     =0000001c
t=75000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=0000001c B(s1)=0000000f P(s2)
     =0000000e i(t1)=00000000 bit(t2)=00000001 limit(t3)=00000008 Result=00000007 ALUResult
     =00000007
t=85000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=0000001c B(s1)=00000007 P(s2)
     =0000000e i(t1)=00000000 bit(t2)=00000001 limit(t3)=00000008 Result=00000001 ALUResult
     =00000001
t=95000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=0000001c B(s1)=00000007 P(s2)
     =0000000e i(t1)=00000001 bit(t2)=00000001 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=105000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=0000001c B(s1)=00000007 P(s2)
     =0000000e i(t1)=00000001 bit(t2)=00000001 limit(t3)=00000008 Result=00000001 ALUResult
     =00000001
```

```
t=115000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=0000001c B(s1)=00000007 P(s2)
     =0000000e i(t1)=00000001 bit(t2)=00000001 limit(t3)=00000008 Result=00000008 ALUResult
     =00000001
t=125000 reset=0 PC=00000014 Instr=00890933 CPUOut=xxxxxxxx A(s0)=0000001c B(s1)=00000007 P(s2)
     =0000000e i(t1)=00000001 bit(t2)=00000001 limit(t3)=00000008 Result=0000002a ALUResult
     =0000002a
t=135000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=0000001c B(s1)=00000007 P(s2)
     =0000002a i(t1)=00000001 bit(t2)=00000001 limit(t3)=00000008 Result=00000038 ALUResult
     =00000038
t=145000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000038 B(s1)=00000007 P(s2)
     =0000002a i(t1)=00000001 bit(t2)=00000001 limit(t3)=00000008 Result=00000003 ALUResult
     =00000003
t=155000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000038 B(s1)=00000003 P(s2)
     =0000002a i(t1)=00000001 bit(t2)=00000001 limit(t3)=00000008 Result=00000002 ALUResult
     =00000002
t=165000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000038 B(s1)=00000003 P(s2)
     =0000002a i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=175000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000038 B(s1)=00000003 P(s2)
     =0000002a i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000001 ALUResult
     =00000001
t=185000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000038 B(s1)=00000003 P(s2)
     =0000002a i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000008 ALUResult
     =00000001
t=195000 reset=0 PC=00000014 Instr=00890933 CPUOut=xxxxxxxx A(s0)=00000038 B(s1)=00000003 P(s2)
     =0000002a i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000062 ALUResult
     =00000062
t=205000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000038 B(s1)=00000003 P(s2)
     =00000062 i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000070 ALUResult
     =00000070
t=215000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000070 B(s1)=00000003 P(s2)
     =00000062 i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000001 ALUResult
     =00000001
t=225000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000070 B(s1)=00000001 P(s2)
     =00000062 i(t1)=00000002 bit(t2)=00000001 limit(t3)=00000008 Result=00000003 ALUResult
     =00000003
t=235000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000070 B(s1)=00000001 P(s2)
     =00000062 i(t1)=00000003 bit(t2)=00000001 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=245000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000070 B(s1)=00000001 P(s2)
     =00000062 i(t1)=00000003 bit(t2)=00000001 limit(t3)=00000008 Result=00000001 ALUResult
     =00000001
t=255000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000070 B(s1)=00000001 P(s2)
     =00000062 i(t1)=00000003 bit(t2)=00000001 limit(t3)=00000008 Result=00000008 ALUResult
     =00000001
t=265000 reset=0 PC=00000014 Instr=00890933 CPUOut=xxxxxxxx A(s0)=00000070 B(s1)=00000001 P(s2)
     =00000062 i(t1)=00000003 bit(t2)=00000001 limit(t3)=00000008 Result=000000d2 ALUResult
     =000000d2
t=275000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000070 B(s1)=00000001 P(s2)
     =000000d2 i(t1)=00000003 bit(t2)=00000001 limit(t3)=00000008 Result=000000e0 ALUResult
     =000000e0
t=285000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=000000e0 B(s1)=00000001 P(s2)
     =000000d2 i(t1)=00000003 bit(t2)=00000001 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=295000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=000000e0 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000003 bit(t2)=00000001 limit(t3)=00000008 Result=00000004 ALUResult
     =00000004
```

```
t=305000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=000000e0 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000004 bit(t2)=00000001 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=315000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=000000e0 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000004 bit(t2)=00000001 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=325000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=000000e0 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=335000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=000000e0 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=000001c0 ALUResult
     =000001c0
t=345000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=000001c0 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=355000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=000001c0 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000004 bit(t2)=00000000 limit(t3)=00000008 Result=00000005 ALUResult
     =00000005
t=365000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=000001c0 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=375000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=000001c0 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=385000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=000001c0 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=395000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=000001c0 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=00000380 ALUResult
     =00000380
t=405000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000380 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=415000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000380 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000005 bit(t2)=00000000 limit(t3)=00000008 Result=00000006 ALUResult
     =00000006
t=425000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000380 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
t=435000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000380 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=445000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000380 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=455000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000380 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000700 ALUResult
     =00000700
t=465000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000700 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=475000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000700 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000006 bit(t2)=00000000 limit(t3)=00000008 Result=00000007 ALUResult
     =00000007
t=485000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000700 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000001
```

```
t=495000 reset=0 PC=0000000c Instr=0014f393 CPUOut=xxxxxxxx A(s0)=00000700 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=505000 reset=0 PC=00000010 Instr=00038463 CPUOut=xxxxxxxx A(s0)=00000700 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000000
t=515000 reset=0 PC=00000018 Instr=00141413 CPUOut=xxxxxxxx A(s0)=00000700 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000e00 ALUResult
     =00000e00
t=525000 reset=0 PC=0000001c Instr=0014d493 CPUOut=xxxxxxxx A(s0)=00000e00 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult
     =00000000
t=535000 reset=0 PC=00000020 Instr=00130313 CPUOut=xxxxxxxx A(s0)=00000e00 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000007 bit(t2)=00000000 limit(t3)=00000008 Result=00000008 ALUResult
     =00000008
t=545000 reset=0 PC=00000024 Instr=ffc344e3 CPUOut=xxxxxxxx A(s0)=00000e00 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=ffffffe8 ALUResult
     =00000000
t=555000 reset=0 PC=00000028 Instr=800002b7 CPUOut=xxxxxxxx A(s0)=00000e00 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=80000000 ALUResult
     =00000000
t=565000 reset=0 PC=0000002c Instr=ffc28293 CPUOut=xxxxxxxx A(s0)=00000e00 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=7ffffffc ALUResult=7
     ffffffc
t=575000 reset=0 PC=00000030 Instr=0122a023 CPUOut=xxxxxxxx A(s0)=00000e00 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=00000000 ALUResult=7
     ffffffc
t=585000 reset=0 PC=00000034 Instr=0000006f CPUOut=000000d2 A(s0)=00000e00 B(s1)=00000000 P(s2)
     =000000d2 i(t1)=00000008 bit(t2)=00000000 limit(t3)=00000008 Result=00000038 ALUResult
     =00000000
```

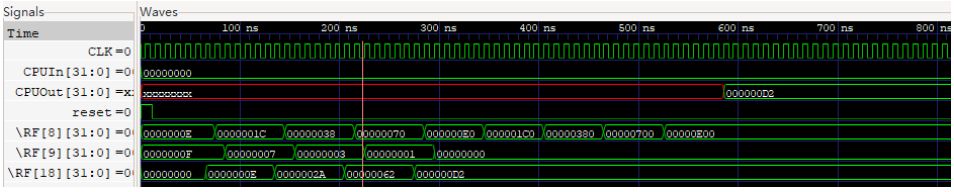


Fig. 17. GTKWave result 3 for Task 2.2.

7.8 analyze

Test Set 1: `output_vcd_1`. The initial register values are `A = 0x00000003` and `B = 0x00000004`. The expected multiplication result is `3 ×4 = 12`, which is `0x0000000C`. In the simulation trace, the final output is `CPUOut = 0000000c`, so this test result is correct.

Test Set 2: `output_vcd_2`. The initial register values are `A = 0x00000000` and `B = 0x00000005`. The expected multiplication result is `0 ×5 = 0`, which is `0x00000000`. The final simulation output is `CPUOut = 00000000`, so this test result is correct.

Test Set 3: `output_vcd_3`. The initial register values are `A = 0x0000000E` and `B = 0x0000000F`. The expected result is `14 ×15 = 210`, which is `0x000000D2`. In the simulation trace, the final output is `CPUOut = 000000d2`, so this test result is correct.

Conclusion. All three Task 2.2 test sets produce the expected multiplication result. The traces also show the correct shift-and-add behaviour: when the least significant bit of `B` is `1`, the current value of `A` is added to `P`; then `A` is shifted left, `B` is shifted right, and the loop counter increases until eight iterations are completed. This confirms that the RISC-V multiplication program functions correctly.

A Appendix

The complete source code for this coursework is available in the following GitHub repository:

https://github.com/xzzc666/myDigitalCoursework

In completing this coursework, AI tools were used as supporting tools for translation, and debugging code issues.